Import Libraries

In [68]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, linear_reset
from scipy.stats import jarque_bera

import warnings
warnings.filterwarnings("ignore")
```

Data Loading

In [69]:
```
df=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\House Sales Price Prediction\Dataset_1\train.csv', low_memory=False)
pd.set_option('display.max_columns', None)
print(df.shape)
df.head()
```

```
(1460, 81)
```

Out[69]:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope	Neighborhoo
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	Inside	Gtl	Collg
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	FR2	Gtl	Veenk
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	Inside	Gtl	Collg
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	Corner	Gtl	Crawf
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	FR2	Gtl	NoRidg

Data Information

In [70]:
```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             1460 non-null   int64
 1   MSSubClass     1460 non-null   int64
 2   MSZoning       1460 non-null   object
 3   LotFrontage    1201 non-null   float64
 4   LotArea        1460 non-null   int64
 5   Street         1460 non-null   object
 6   Alley          91 non-null     object
 7   LotShape       1460 non-null   object
 8   LandContour    1460 non-null   object
 9   Utilities      1460 non-null   object
 10  LotConfig      1460 non-null   object
 11  LandSlope      1460 non-null   object
 12  Neighborhood   1460 non-null   object
 13  Condition1     1460 non-null   object
 14  Condition2     1460 non-null   object
 15  BldgType       1460 non-null   object
 16  HouseStyle     1460 non-null   object
 17  OverallQual    1460 non-null   int64
 18  OverallCond    1460 non-null   int64
 19  YearBuilt      1460 non-null   int64
 20  YearRemodAdd   1460 non-null   int64
 21  RoofStyle      1460 non-null   object
 22  RoofMatl       1460 non-null   object
 23  Exterior1st    1460 non-null   object
 24  Exterior2nd    1460 non-null   object
 25  MasVnrType     588 non-null    object
 26  MasVnrArea     1452 non-null   float64
 27  ExterQual      1460 non-null   object
 28  ExterCond      1460 non-null   object
 29  Foundation     1460 non-null   object
 30  BsmtQual       1423 non-null   object
 31  BsmtCond       1423 non-null   object
 32  BsmtExposure   1422 non-null   object
 33  BsmtFinType1   1423 non-null   object
 34  BsmtFinSF1     1460 non-null   int64
 35  BsmtFinType2   1422 non-null   object
```

```
 36  BsmtFinSF2     1460 non-null   int64
 37  BsmtUnfSF      1460 non-null   int64
 38  TotalBsmtSF    1460 non-null   int64
 39  Heating        1460 non-null   object
 40  HeatingQC      1460 non-null   object
 41  CentralAir     1460 non-null   object
 42  Electrical     1459 non-null   object
 43  1stFlrSF       1460 non-null   int64
 44  2ndFlrSF       1460 non-null   int64
 45  LowQualFinSF   1460 non-null   int64
 46  GrLivArea      1460 non-null   int64
 47  BsmtFullBath   1460 non-null   int64
 48  BsmtHalfBath   1460 non-null   int64
 49  FullBath       1460 non-null   int64
 50  HalfBath       1460 non-null   int64
 51  BedroomAbvGr   1460 non-null   int64
 52  KitchenAbvGr   1460 non-null   int64
 53  KitchenQual    1460 non-null   object
 54  TotRmsAbvGrd   1460 non-null   int64
 55  Functional     1460 non-null   object
 56  Fireplaces     1460 non-null   int64
 57  FireplaceQu    770 non-null    object
 58  GarageType     1379 non-null   object
 59  GarageYrBlt    1379 non-null   float64
 60  GarageFinish   1379 non-null   object
 61  GarageCars     1460 non-null   int64
 62  GarageArea     1460 non-null   int64
 63  GarageQual     1379 non-null   object
 64  GarageCond     1379 non-null   object
 65  PavedDrive     1460 non-null   object
 66  WoodDeckSF     1460 non-null   int64
 67  OpenPorchSF    1460 non-null   int64
 68  EnclosedPorch  1460 non-null   int64
 69  3SsnPorch      1460 non-null   int64
 70  ScreenPorch    1460 non-null   int64
 71  PoolArea       1460 non-null   int64
 72  PoolQC         7 non-null      object
 73  Fence          281 non-null    object
 74  MiscFeature    54 non-null     object
 75  MiscVal        1460 non-null   int64
 76  MoSold         1460 non-null   int64
```

```
 77  YrSold         1460 non-null   int64
 78  SaleType       1460 non-null   object
 79  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

Out[70]:

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtI
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	1460.0
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	443.6
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	456.0
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	0.0
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	0.0
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	383.5
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	712.2
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	5644.0

Check data types

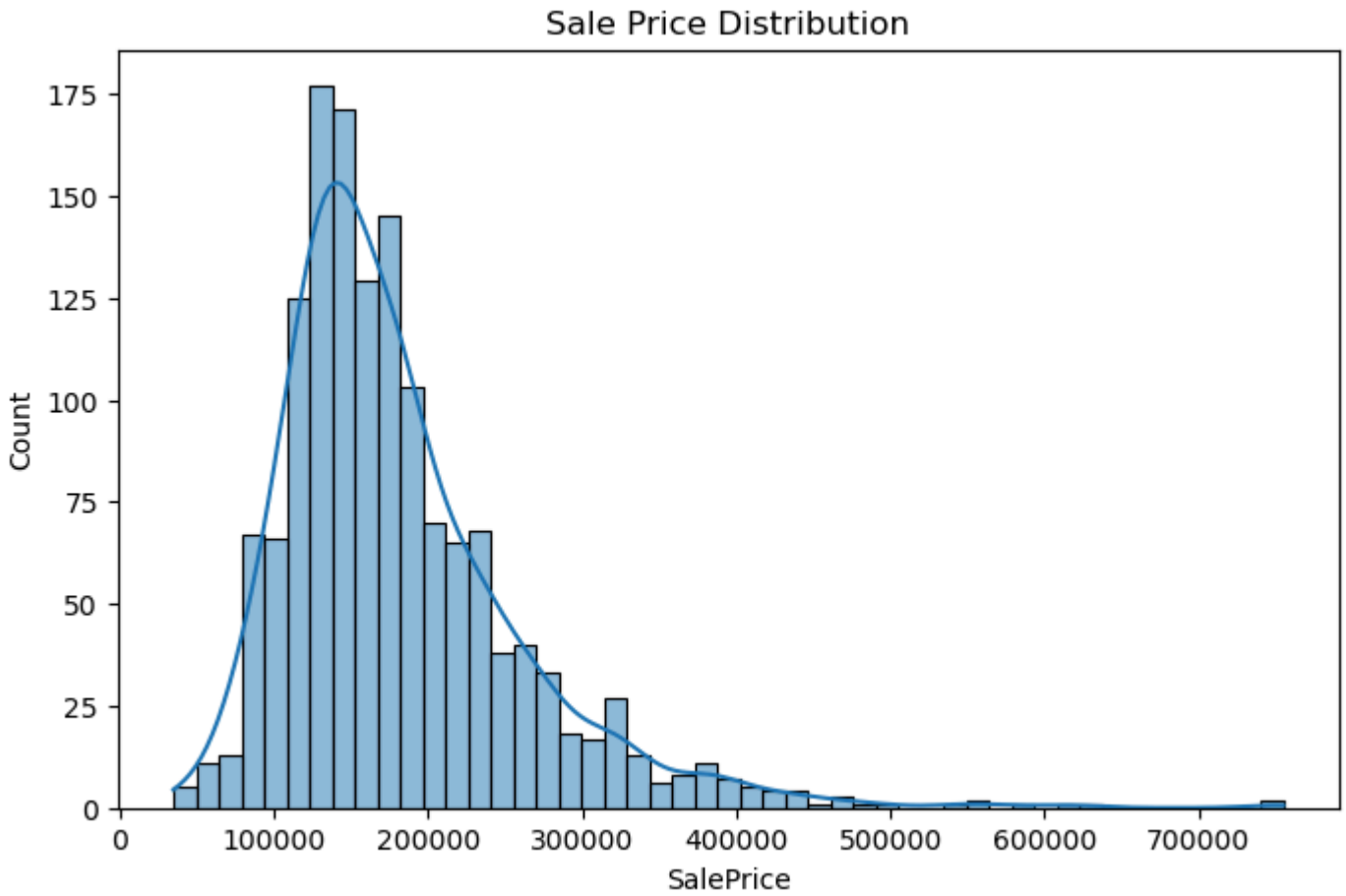
In [71]:
```
df.dtypes.value_counts()
```

Out[71]:
```
object     43
int64      35
float64     3
Name: count, dtype: int64
```

Target Variable Analysis

In [72]:
```
plt.figure(figsize=(8,5))
sns.histplot(df["SalePrice"], kde=True)
```

```
plt.title("Sale Price Distribution")
plt.show()
```



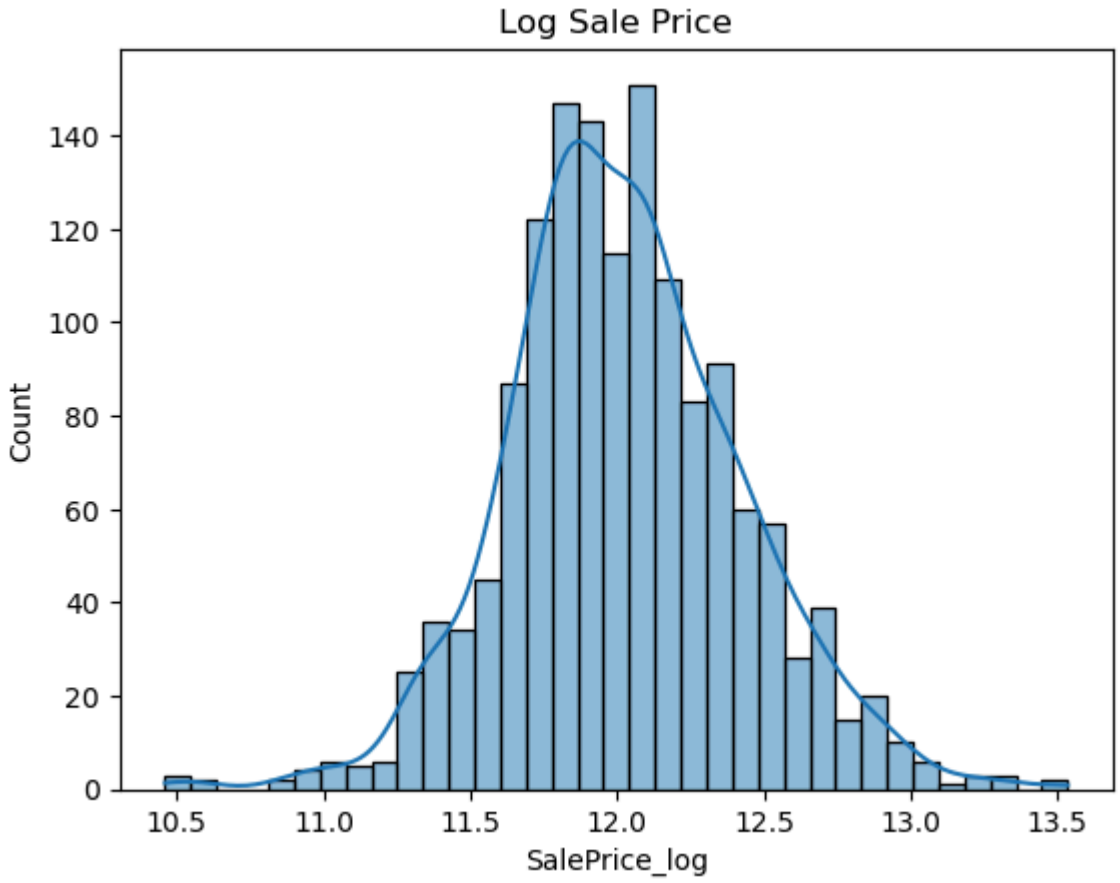
Check skewness

In [73]:
```
print("Skewness:", df["SalePrice"].skew())
```

```
Skewness: 1.8828757597682129
```

Log transformation

In [74]:
```
df["SalePrice_log"] = np.log(df["SalePrice"])
sns.histplot(df["SalePrice_log"], kde=True)
plt.title("Log Sale Price")
plt.show()
```



Define Variable Types

In [75]:
```
num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(include="object").columns.tolist()
```

In [76]:
```
df.dtypes.value_counts()
```

Out[76]:
```
object     43
int64      35
float64     4
Name: count, dtype: int64
```

Recalculate column types

In [77]:
```
num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(include="object").columns.tolist()
```

Remove target variable

In [78]:
```
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

Missing Value Imputation

In [79]:
```
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

In [80]:
```
cat_cols = df.select_dtypes(include="object").columns
for col in cat_cols:
    print(col, df[col].nunique())
```

```
MSZoning 5
Street 2
Alley 2
LotShape 4
LandContour 4
Utilities 2
LotConfig 5
LandSlope 3
Neighborhood 25
Condition1 9
Condition2 8
BldgType 5
HouseStyle 8
RoofStyle 6
RoofMatl 8
Exterior1st 15
Exterior2nd 16
MasVnrType 3
ExterQual 4
ExterCond 5
Foundation 6
BsmtQual 4
BsmtCond 4
BsmtExposure 4
BsmtFinType1 6
BsmtFinType2 6
Heating 6
HeatingQC 5
CentralAir 2
Electrical 5
KitchenQual 4
Functional 7
FireplaceQu 5
GarageType 6
GarageFinish 3
GarageQual 5
GarageCond 5
PavedDrive 3
PoolQC 3
Fence 4
MiscFeature 4
```

```
SaleType 9
SaleCondition 6
```

Categorical Vairable Feature Engineering

Reduce Exterior Types

In [81]:
```
top_ext = df["Exterior1st"].value_counts().index[:5]

df["Exterior_grp_1"] = df["Exterior1st"].apply(
    lambda x: x if x in top_ext else "Other"
)
```

In [82]:
```
top_ext = df["Exterior2nd"].value_counts().index[:5]

df["Exterior_grp_2"] = df["Exterior2nd"].apply(
    lambda x: x if x in top_ext else "Other"
)
```

In [83]:
```
cat_cols = df.select_dtypes(include="object").columns
for col in cat_cols:
    print(col, df[col].nunique())
```

```
MSZoning 5
Street 2
Alley 2
LotShape 4
LandContour 4
Utilities 2
LotConfig 5
LandSlope 3
Neighborhood 25
Condition1 9
Condition2 8
BldgType 5
HouseStyle 8
RoofStyle 6
RoofMatl 8
Exterior1st 15
Exterior2nd 16
MasVnrType 3
ExterQual 4
ExterCond 5
Foundation 6
BsmtQual 4
BsmtCond 4
BsmtExposure 4
BsmtFinType1 6
BsmtFinType2 6
Heating 6
HeatingQC 5
CentralAir 2
Electrical 5
KitchenQual 4
Functional 7
FireplaceQu 5
GarageType 6
GarageFinish 3
GarageQual 5
GarageCond 5
PavedDrive 3
PoolQC 3
Fence 4
MiscFeature 4
```

```
SaleType 9
SaleCondition 6
Exterior_grp_1 6
Exterior_grp_2 6
```

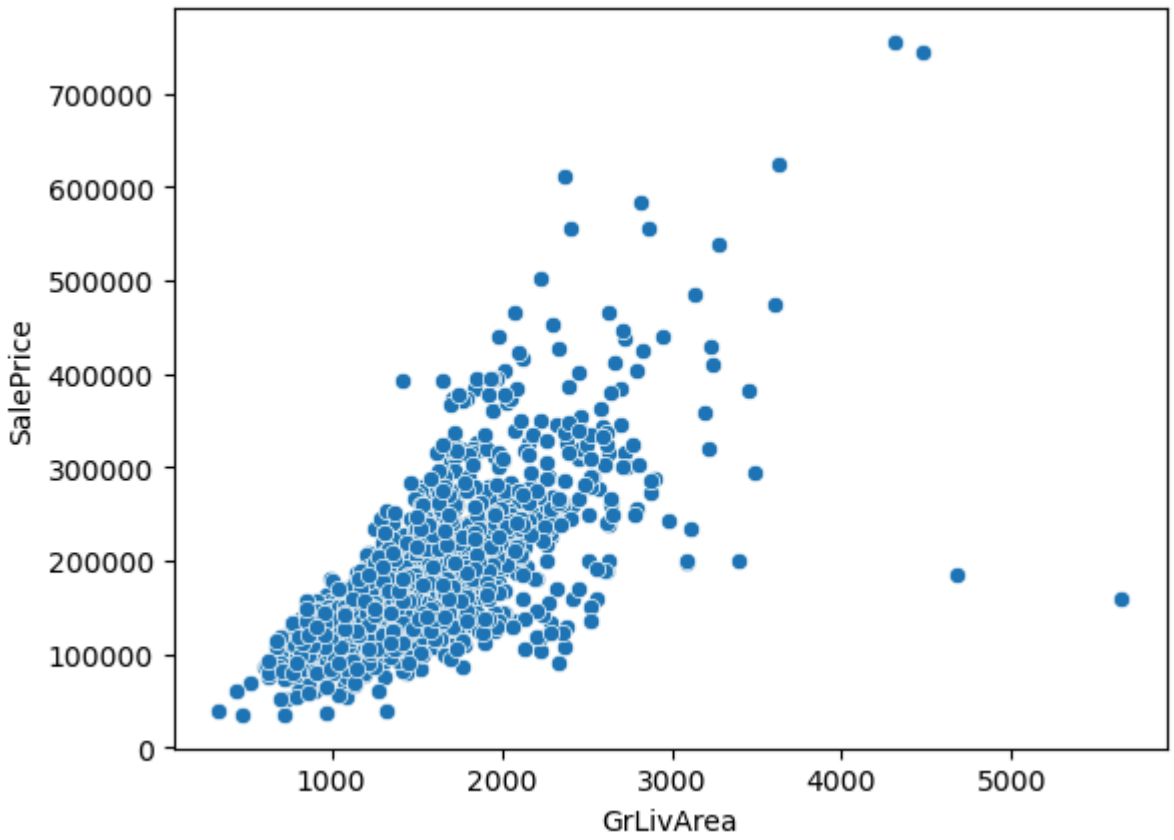
Reduce Neighborhood Levels

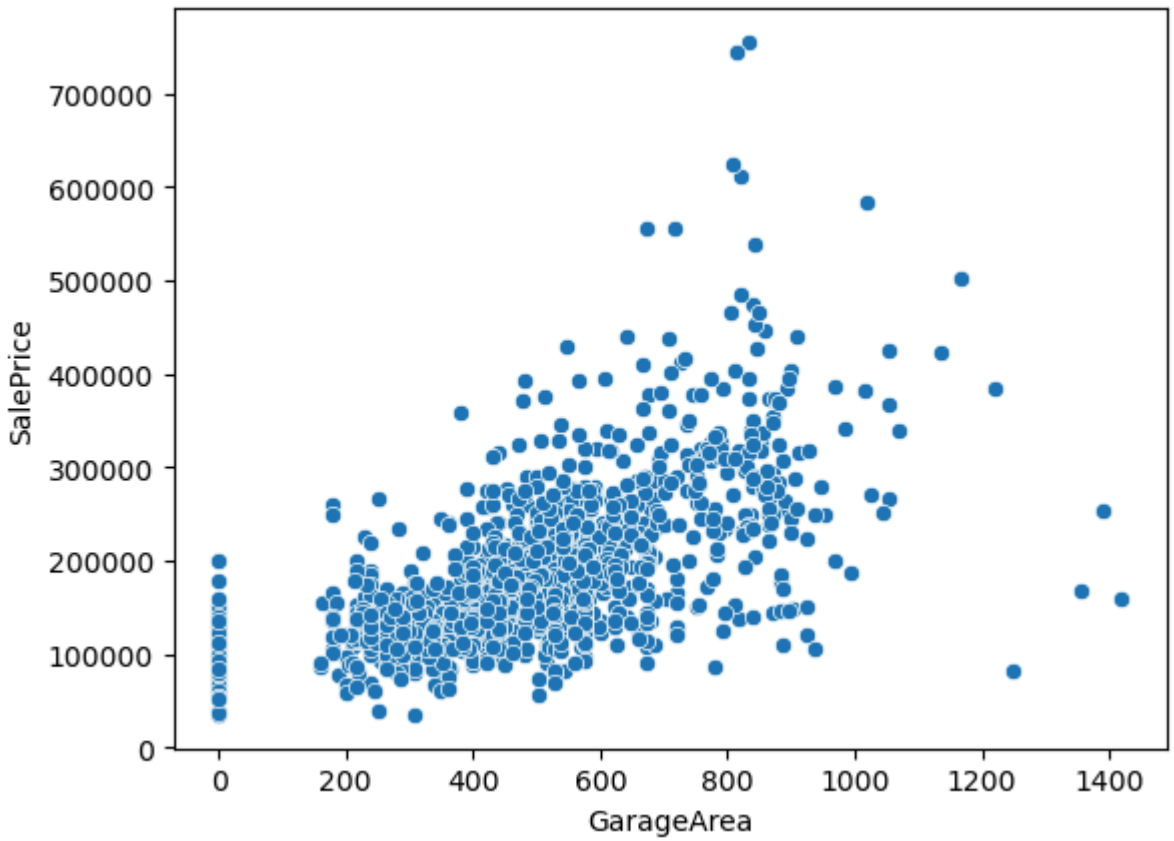
In [84]:
```
top_neigh = df["Neighborhood"].value_counts().index[:8]

df["Neighborhood_grp"] = df["Neighborhood"].apply(
    lambda x: x if x in top_neigh else "Other"
)
```

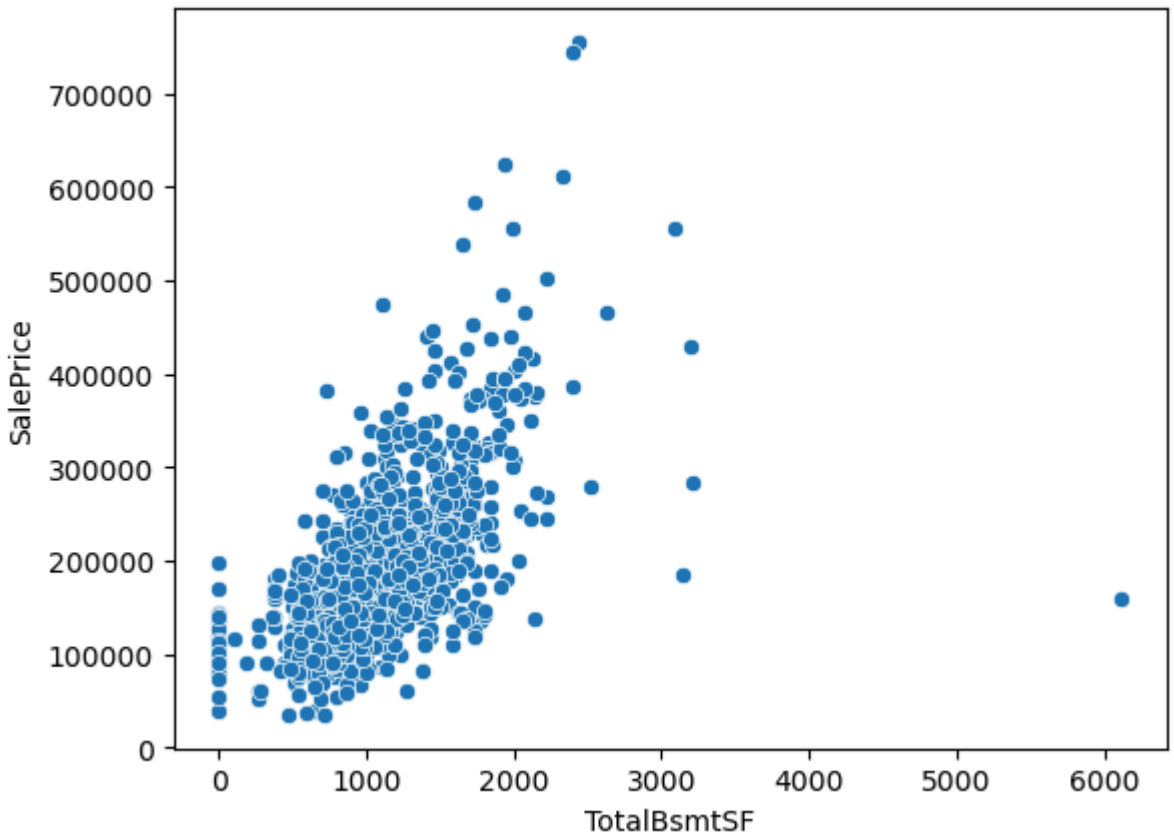
Exploratory Analysis

Scatter Plot

In [85]:
```
features = ["GrLivArea","GarageArea","TotalBsmtSF"]

for f in features:
    sns.scatterplot(x=df[f], y=df["SalePrice"])
    plt.show()
```

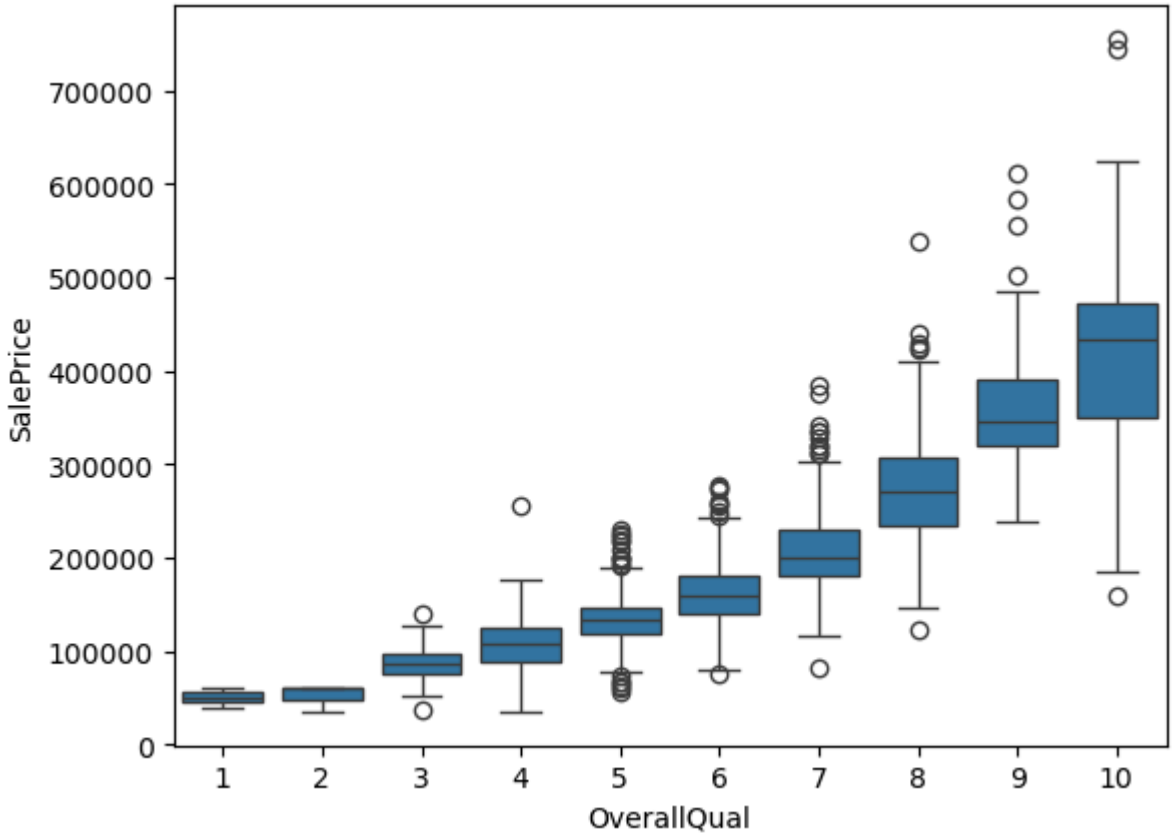

700000
600000
500000
400000
300000
200000
100000
0
SalePrice
1000
2000
3000
4000
5000
GrLivArea


700000
600000
500000
400000
300000
200000
100000
0
SalePrice
0
200
400
600
800
1000
1200
1400
GarageArea



Boxplot for categorical variables

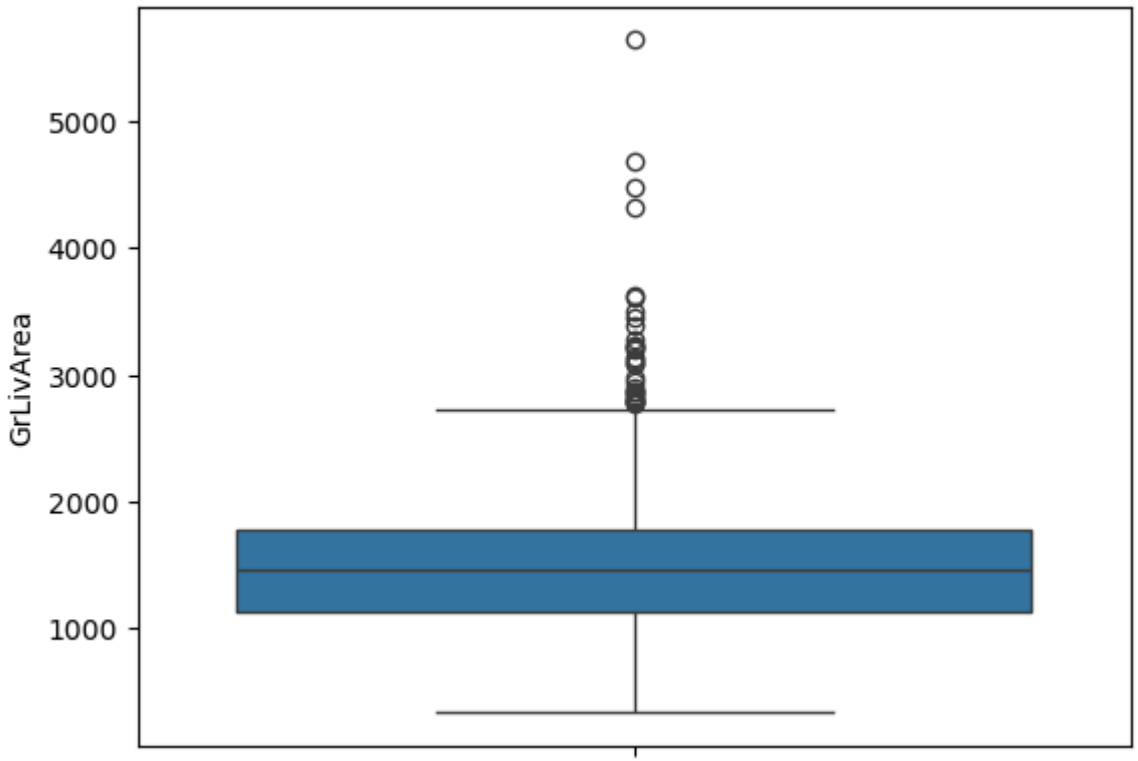
In [86]:
```
sns.boxplot(x=df["OverallQual"], y=df["SalePrice"])
plt.show()
```



Outlier Analysis

Boxplot

In [87]:
```
sns.boxplot(df["GrLivArea"])
plt.show()
```



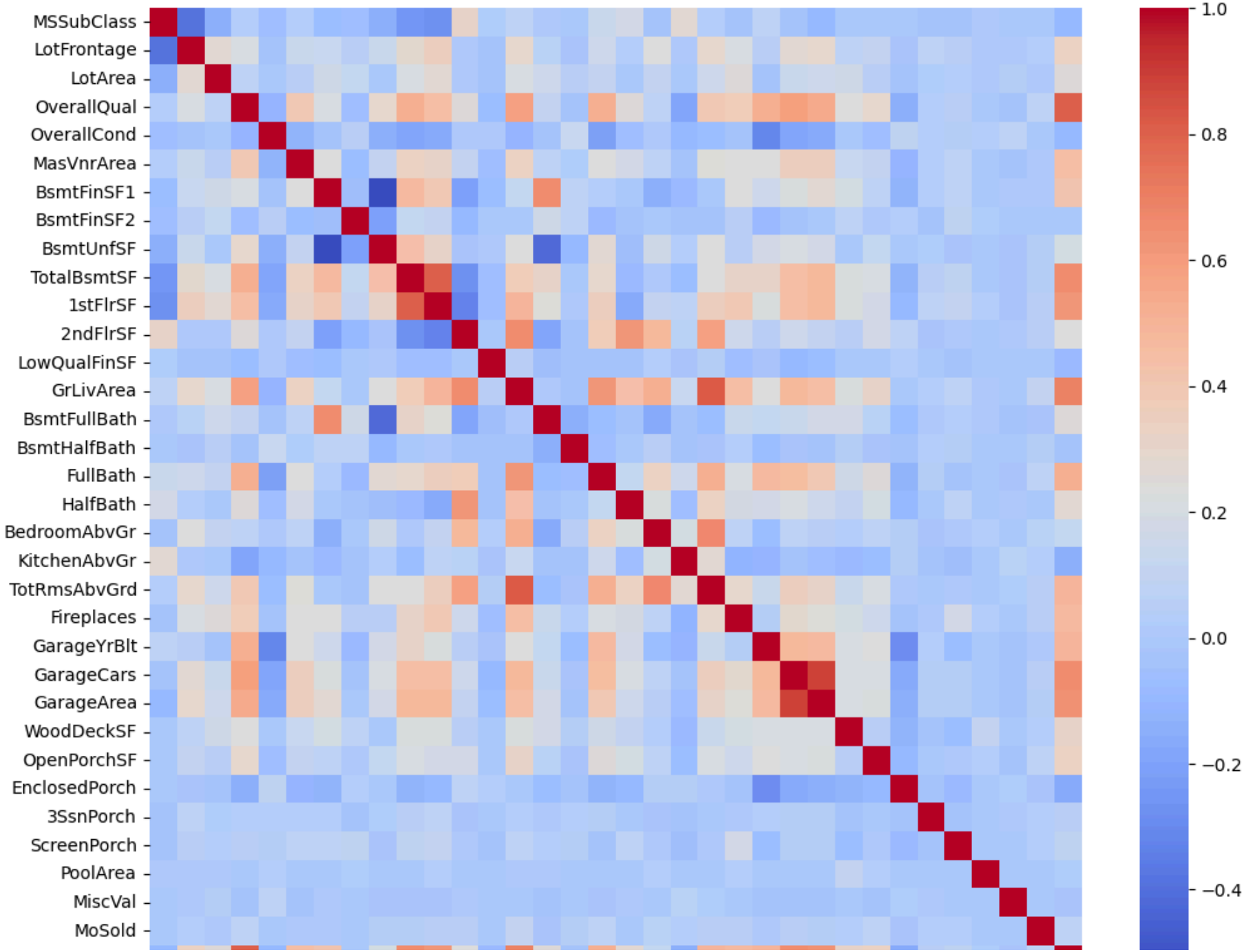
Outlier Detection (IQR Method)

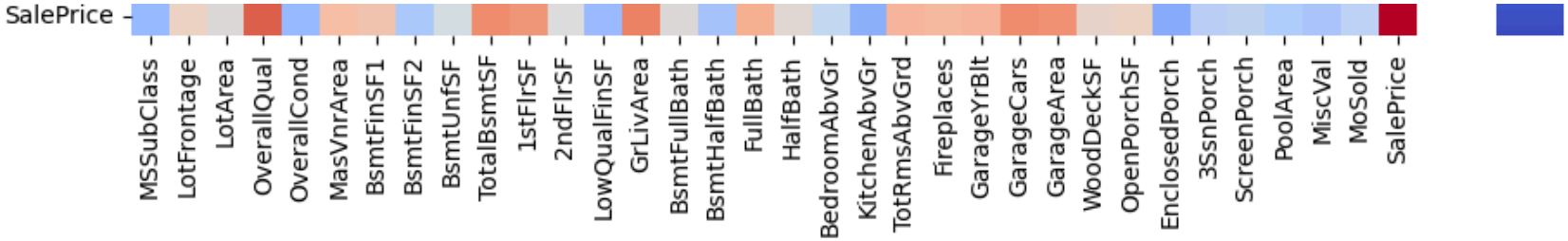
In [88]:
```
Q1 = df["GrLivArea"].quantile(0.25)
Q3 = df["GrLivArea"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

df = df[(df["GrLivArea"] > lower) & (df["GrLivArea"] < upper)]
```

In [89]:
```
X = df.drop(["SalePrice_log","Id","YearBuilt","YearRemodAdd","YrSold"], axis=1)

y = df["SalePrice_log"]
```

Correlation Analysis

In [90]:
```
numeric_X = X.select_dtypes(include=np.number)
corr = numeric_X.corr()

plt.figure(figsize=(12,10))
sns.heatmap(corr, cmap="coolwarm")
plt.show()
```


MSSubClass
LotFrontage
LotArea
OverallQual
OverallCond
MasVnrArea
BsmtFinSF1
BsmtFinSF2
BsmtUnfSF
TotalBsmtSF
1stFlrSF
2ndFlrSF
LowQualFinSF
GrLivArea
BsmtFullBath
BsmtHalfBath
FullBath
HalfBath
BedroomAbvGr
KitchenAbvGr
TotRmsAbvGrd
Fireplaces
GarageYrBlt
GarageCars
GarageArea
WoodDeckSF
OpenPorchSF
EnclosedPorch
3SsnPorch
ScreenPorch
PoolArea
MiscVal
MoSold
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4



Top Correlation with Target

In [91]:
```
corr_target = corr["SalePrice"].sort_values(ascending=False)
corr_target.head(15)
```

Out[91]:
```
SalePrice       1.000000
OverallQual     0.804186
GrLivArea       0.696162
GarageCars      0.654766
TotalBsmtSF     0.648681
GarageArea      0.638838
1stFlrSF        0.615310
FullBath        0.528040
GarageYrBlt     0.496169
TotRmsAbvGrd    0.495099
Fireplaces      0.465247
MasVnrArea      0.442265
BsmtFinSF1      0.416804
LotFrontage     0.334599
OpenPorchSF     0.330455
Name: SalePrice, dtype: float64
```

In [92]:
```
X = df.drop("SalePrice", axis=1)
```

Re-Define X and y

In [93]:
```
X = df.drop(["SalePrice", "SalePrice_log", "Id"], axis=1)
y = df["SalePrice_log"]
```

Train-Test Split

In [94]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (1143, 82)
Test shape: (286, 82)
```

Keep only numeric variables

In [95]:
```
X_num = X_train.select_dtypes(include=np.number)
```

VIF Calulation and High VIF removal Function

In [96]:
```
def calculate_vif(X, threshold=5):

    while True:

        vif = pd.DataFrame()
        vif["Variable"] = X.columns
        vif["VIF"] = [
            variance_inflation_factor(X.values, i)
            for i in range(X.shape[1])
        ]

        max_vif = vif["VIF"].max()

        if max_vif > threshold:

            drop_var = vif.sort_values("VIF", ascending=False)["Variable"].iloc[0]
```

```
            if drop_var == "const":
                break

            print("Dropping:", drop_var, "VIF:", max_vif)

            X = X.drop(columns=[drop_var])

        else:
            break

    return X
```

Run VIF Function

In [97]:
```
# Add constant
X_num = sm.add_constant(X_num)
# Run VIF
X_vif = calculate_vif(X_num, threshold=5)
print("Remaining variables after VIF:")
print(X_vif.columns)
```

```
Dropping: LowQualFinSF VIF: inf
Dropping: BsmtFinSF1 VIF: inf
Remaining variables after VIF:
Index(['const', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual',
       'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea', 'BsmtFinSF2',
       'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'GrLivArea',
       'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath', 'BedroomAbvGr',
       'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageYrBlt',
       'GarageCars', 'GarageArea', 'WoodDeckSF', 'OpenPorchSF',
       'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal',
       'MoSold', 'YrSold'],
      dtype='object')
```

Convert categorical variables

In [98]:
```
X_cat = pd.get_dummies(X_train, drop_first=True).astype(int)
```

In [99]:
```
X = pd.concat([X_vif, X_cat], axis=1)
```

Add constant

In [100…
```
X = sm.add_constant(X)
```

Stepwise Multiple Regression (Backward Elimination)

In [101…
```
X_model = sm.add_constant(X)

def backward_elimination(X, y, significance_level=0.05):

    X = X.copy()

    while True:

        model = sm.OLS(y, X).fit()

        # drop constant safely
        pvalues = model.pvalues.drop("const", errors="ignore")

        max_p = pvalues.max()

        if max_p > significance_level:

            drop_var = pvalues.idxmax()

            print(f"Dropping {drop_var} (p-value = {max_p:.4f})")

            X = X.drop(columns=[drop_var])

        else:
            break

    model = sm.OLS(y, X).fit()

    return model, X
```

```
# Run stepwise regression
model, X_final = backward_elimination(X_model, y_train)

print(model.summary())

print("\n Model Summary")
print(model.summary())
```

```
Dropping SaleType_WD (p-value = 0.9865)
Dropping Fence_MnWw (p-value = 0.9819)
Dropping BsmtFinType2_LwQ (p-value = 0.9783)
Dropping RoofMatl_WdShake (p-value = 0.9761)
Dropping Exterior1st_Stone (p-value = 0.9742)
Dropping Exterior2nd_Stone (p-value = 0.9775)
Dropping GarageCond_TA (p-value = 0.9696)
Dropping RoofStyle_Hip (p-value = 0.9549)
Dropping BldgType_TwnhsE (p-value = 0.9513)
Dropping Exterior1st_ImStucc (p-value = 0.9534)
Dropping HouseStyle_SFoyer (p-value = 0.9354)
Dropping HouseStyle_SLvl (p-value = 0.9572)
Dropping SaleType_ConLI (p-value = 0.9356)
Dropping LotConfig_FR3 (p-value = 0.9301)
Dropping RoofMatl_Roll (p-value = 0.9277)
Dropping Exterior1st_WdShing (p-value = 0.9130)
Dropping RoofStyle_Gable (p-value = 0.9147)
Dropping MiscVal (p-value = 0.8955)
Dropping Condition2_RRAn (p-value = 0.8675)
Dropping HouseStyle_1.5Unf (p-value = 0.8444)
Dropping MiscFeature_Shed (p-value = 0.8478)
Dropping MoSold (p-value = 0.8494)
Dropping Neighborhood_NoRidge (p-value = 0.8430)
Dropping BedroomAbvGr (p-value = 0.8504)
Dropping Heating_OthW (p-value = 0.8179)
Dropping BsmtExposure_No (p-value = 0.8193)
Dropping SaleType_ConLw (p-value = 0.8011)
Dropping Neighborhood_NPkVill (p-value = 0.7888)
Dropping RoofMatl_WdShngl (p-value = 0.7774)
Dropping ExterCond_Po (p-value = 0.7554)
Dropping Neighborhood_SWISU (p-value = 0.7545)
Dropping YrSold (p-value = 0.7448)
Dropping Neighborhood_BrkSide (p-value = 0.7451)
Dropping Neighborhood_Blueste (p-value = 0.7703)
Dropping RoofMatl_Tar&Grv (p-value = 0.7242)
Dropping Heating_Wall (p-value = 0.7160)
Dropping LotShape_IR3 (p-value = 0.7186)
Dropping ExterCond_TA (p-value = 0.6893)
Dropping FireplaceQu_TA (p-value = 0.6830)
Dropping Exterior1st_CBlock (p-value = 0.6873)
Dropping Exterior2nd_CBlock (p-value = 0.6873)
```

```
Dropping ExterQual_TA (p-value = 0.6844)
Dropping 1stFlrSF (p-value = 0.6806)
Dropping 2ndFlrSF (p-value = 0.9681)
Dropping BldgType_Duplex (p-value = 0.6626)
Dropping LandContour_HLS (p-value = 0.6683)
Dropping GarageFinish_RFn (p-value = 0.6204)
Dropping LandContour_Lvl (p-value = 0.5970)
Dropping BsmtFinType2_Unf (p-value = 0.6169)
Dropping ExterQual_Gd (p-value = 0.6041)
Dropping Exterior2nd_Other (p-value = 0.5780)
Dropping OpenPorchSF (p-value = 0.5791)
Dropping ExterCond_Fa (p-value = 0.5937)
Dropping GarageQual_Gd (p-value = 0.5834)
Dropping BsmtExposure_Mn (p-value = 0.5698)
Dropping GarageCond_Po (p-value = 0.5810)
Dropping BsmtFinType2_GLQ (p-value = 0.5642)
Dropping FireplaceQu_Po (p-value = 0.5623)
Dropping RoofStyle_Gambrel (p-value = 0.5528)
Dropping Electrical_SBrkr (p-value = 0.5292)
Dropping Foundation_Slab (p-value = 0.5277)
Dropping SaleCondition_Alloca (p-value = 0.5124)
Dropping FireplaceQu_Fa (p-value = 0.5070)
Dropping BsmtHalfBath (p-value = 0.5149)
Dropping SaleType_New (p-value = 0.4924)
Dropping FireplaceQu_Gd (p-value = 0.4946)
Dropping Condition1_Feedr (p-value = 0.4732)
Dropping Condition1_PosA (p-value = 0.5685)
Dropping Condition1_RRAn (p-value = 0.4516)
Dropping LowQualFinSF (p-value = 0.4566)
Dropping GarageCond_Gd (p-value = 0.4356)
Dropping Exterior2nd_Wd Shng (p-value = 0.4117)
Dropping Exterior2nd_BrkFace (p-value = 0.4923)
Dropping Exterior2nd_ImStucc (p-value = 0.4962)
Dropping Exterior2nd_AsphShn (p-value = 0.5238)
Dropping Exterior2nd_Brk Cmn (p-value = 0.4732)
Dropping Exterior2nd_Stucco (p-value = 0.4541)
Dropping GarageQual_Po (p-value = 0.4456)
Dropping Electrical_FuseP (p-value = 0.5978)
Dropping SaleCondition_AdjLand (p-value = 0.4585)
Dropping HeatingQC_Fa (p-value = 0.4278)
Dropping Neighborhood_ClearCr (p-value = 0.4119)
```

```
Dropping MasVnrType_BrkFace (p-value = 0.4067)
Dropping GarageType_CarPort (p-value = 0.3997)
Dropping GarageType_Basment (p-value = 0.5037)
Dropping GarageType_BuiltIn (p-value = 0.7389)
Dropping Neighborhood_Veenker (p-value = 0.4032)
Dropping Fence_GdWo (p-value = 0.4000)
Dropping HeatingQC_Po (p-value = 0.3972)
Dropping BsmtFinType1_Unf (p-value = 0.3843)
Dropping LandSlope_Mod (p-value = 0.3507)
Dropping TotRmsAbvGrd (p-value = 0.3684)
Dropping Exterior1st_CemntBd (p-value = 0.3070)
Dropping MasVnrArea (p-value = 0.2883)
Dropping GarageCond_Fa (p-value = 0.2815)
Dropping BsmtFinType1_BLQ (p-value = 0.2824)
Dropping 3SsnPorch (p-value = 0.2703)
Dropping Condition1_RRNe (p-value = 0.3164)
Dropping Utilities_NoSeWa (p-value = 0.2470)
Dropping Electrical_FuseF (p-value = 0.2280)
Dropping Foundation_CBlock (p-value = 0.2244)
Dropping BsmtFinType1_Rec (p-value = 0.2034)
Dropping Neighborhood_IDOTRR (p-value = 0.2193)
Dropping PavedDrive_P (p-value = 0.2240)
Dropping BsmtFinType1_LwQ (p-value = 0.2027)
Dropping GarageType_Attchd (p-value = 0.2108)
Dropping GarageType_Detchd (p-value = 0.2905)
Dropping Condition1_RRNn (p-value = 0.1921)
Dropping LotShape_Reg (p-value = 0.2019)
Dropping Street_Pave (p-value = 0.1961)
Dropping LotShape_IR2 (p-value = 0.2022)
Dropping HouseStyle_2Story (p-value = 0.1952)
Dropping SaleType_Con (p-value = 0.1768)
Dropping GarageArea (p-value = 0.1695)
Dropping EnclosedPorch (p-value = 0.1538)
Dropping BsmtFinType2_Rec (p-value = 0.1414)
Dropping GarageQual_TA (p-value = 0.1913)
Dropping Condition2_RRNn (p-value = 0.2041)
Dropping Condition2_Norm (p-value = 0.3160)
Dropping Exterior2nd_CmentBd (p-value = 0.1648)
Dropping Alley_Pave (p-value = 0.1556)
Dropping Condition2_Feedr (p-value = 0.1576)
Dropping BsmtFinSF2 (p-value = 0.1465)
```

```
Dropping BsmtFinSF1 (p-value = 0.1810)
Dropping GarageYrBlt (p-value = 0.1414)
Dropping FullBath (p-value = 0.1371)
Dropping HalfBath (p-value = 0.2835)
Dropping Neighborhood_BrDale (p-value = 0.1658)
Dropping Functional_Min1 (p-value = 0.1881)
Dropping Functional_Min2 (p-value = 0.2857)
Dropping LotConfig_FR2 (p-value = 0.1583)
Dropping BsmtQual_Fa (p-value = 0.1635)
Dropping BsmtQual_TA (p-value = 0.3114)
Dropping BsmtQual_Gd (p-value = 0.1807)
Dropping Condition1_RRAe (p-value = 0.1401)
Dropping LandContour_Low (p-value = 0.1219)
Dropping Electrical_Mix (p-value = 0.1576)
Dropping RoofStyle_Mansard (p-value = 0.1445)
Dropping Neighborhood_Timber (p-value = 0.1430)
Dropping ExterQual_Fa (p-value = 0.1167)
Dropping SaleCondition_Family (p-value = 0.1064)
Dropping SaleType_ConLD (p-value = 0.1062)
Dropping LotConfig_Inside (p-value = 0.0925)
Dropping Exterior1st_Stucco (p-value = 0.0739)
Dropping MasVnrType_Stone (p-value = 0.0853)
Dropping Neighborhood_NWAmes (p-value = 0.0801)
Dropping Neighborhood_SawyerW (p-value = 0.1309)
Dropping HouseStyle_1Story (p-value = 0.0635)
Dropping GarageFinish_Unf (p-value = 0.0686)
Dropping Condition1_PosN (p-value = 0.0719)
Dropping HeatingQC_Gd (p-value = 0.0587)
Dropping PoolQC_Gd (p-value = 0.0507)
Dropping ExterCond_Gd (p-value = 0.0576)
                            OLS Regression Results
==============================================================================
Dep. Variable:          SalePrice_log   R-squared:                       0.941
Model:                            OLS   Adj. R-squared:                  0.937
Method:                 Least Squares   F-statistic:                     207.9
Date:                Sun, 15 Mar 2026   Prob (F-statistic):               0.00
Time:                        22:42:15   Log-Likelihood:                 1087.6
No. Observations:                1143   AIC:                            -2009.
Df Residuals:                    1060   BIC:                            -1591.
Df Model:                          82
Covariance Type:            nonrobust
```

	coef	std err	t	P>|t|	[0.025	0.975]
const	3.0527	0.349	8.748	0.000	2.368	3.737
MSSubClass	-0.0002	5.8e-05	-2.763	0.006	-0.000	-4.65e-05
LotFrontage	0.0003	9.35e-05	2.914	0.004	8.9e-05	0.000
LotArea	1.391e-06	2.28e-07	6.112	0.000	9.44e-07	1.84e-06
OverallQual	0.0230	0.002	10.683	0.000	0.019	0.027
OverallCond	0.0171	0.002	9.215	0.000	0.013	0.021
YearBuilt	0.0008	0.000	6.595	0.000	0.001	0.001
YearRemodAdd	0.0005	0.000	3.769	0.000	0.000	0.001
BsmtUnfSF	-2.876e-05	5.53e-06	-5.205	0.000	-3.96e-05	-1.79e-05
TotalBsmtSF	7.281e-05	6.64e-06	10.963	0.000	5.98e-05	8.58e-05
GrLivArea	0.0002	5.28e-06	28.607	0.000	0.000	0.000
BsmtFullBath	0.0131	0.004	3.149	0.002	0.005	0.021
KitchenAbvGr	-0.0464	0.008	-5.592	0.000	-0.063	-0.030
Fireplaces	0.0096	0.003	3.221	0.001	0.004	0.015
GarageCars	0.0232	0.003	8.209	0.000	0.018	0.029
WoodDeckSF	5.346e-05	1.31e-05	4.080	0.000	2.77e-05	7.92e-05
ScreenPorch	7.93e-05	2.97e-05	2.668	0.008	2.1e-05	0.000
PoolArea	0.0002	4.46e-05	3.370	0.001	6.28e-05	0.000
MSSubClass	-0.0002	5.8e-05	-2.763	0.006	-0.000	-4.65e-05
LotFrontage	0.0003	9.35e-05	2.914	0.004	8.9e-05	0.000
LotArea	1.391e-06	2.28e-07	6.113	0.000	9.45e-07	1.84e-06
OverallQual	0.0230	0.002	10.683	0.000	0.019	0.027
OverallCond	0.0171	0.002	9.215	0.000	0.013	0.021
YearBuilt	0.0008	0.000	6.595	0.000	0.001	0.001
YearRemodAdd	0.0005	0.000	3.769	0.000	0.000	0.001
BsmtUnfSF	-2.876e-05	5.53e-06	-5.205	0.000	-3.96e-05	-1.79e-05
TotalBsmtSF	7.281e-05	6.64e-06	10.963	0.000	5.98e-05	8.58e-05
GrLivArea	0.0002	5.28e-06	28.607	0.000	0.000	0.000
BsmtFullBath	0.0131	0.004	3.149	0.002	0.005	0.021
KitchenAbvGr	-0.0464	0.008	-5.592	0.000	-0.063	-0.030
Fireplaces	0.0096	0.003	3.221	0.001	0.004	0.015
GarageCars	0.0232	0.003	8.209	0.000	0.018	0.029
WoodDeckSF	5.346e-05	1.31e-05	4.080	0.000	2.77e-05	7.92e-05
ScreenPorch	7.93e-05	2.97e-05	2.668	0.008	2.1e-05	0.000
PoolArea	0.0002	4.46e-05	3.370	0.001	6.28e-05	0.000
MSZoning_FV	0.3964	0.047	8.417	0.000	0.304	0.489
MSZoning_RH	0.3703	0.048	7.755	0.000	0.277	0.464
MSZoning_RL	0.3757	0.038	9.802	0.000	0.300	0.451

MSZoning_RM	0.3362	0.039	8.690	0.000	0.260	0.412
LotConfig_CulDSac	0.0475	0.013	3.682	0.000	0.022	0.073
LandSlope_Sev	-0.2816	0.057	-4.964	0.000	-0.393	-0.170
Neighborhood_CollgCr	0.5493	0.065	8.398	0.000	0.421	0.678
Neighborhood_Crawfor	0.1176	0.018	6.564	0.000	0.082	0.153
Neighborhood_Edwards	0.2567	0.032	8.071	0.000	0.194	0.319
Neighborhood_Gilbert	0.2705	0.034	8.030	0.000	0.204	0.337
Neighborhood_MeadowV	-0.1258	0.034	-3.697	0.000	-0.193	-0.059
Neighborhood_Mitchel	-0.0478	0.018	-2.658	0.008	-0.083	-0.013
Neighborhood_NAmes	0.2711	0.031	8.619	0.000	0.209	0.333
Neighborhood_NridgHt	0.3133	0.033	9.567	0.000	0.249	0.378
Neighborhood_OldTown	0.2647	0.031	8.473	0.000	0.203	0.326
Neighborhood_Sawyer	0.2697	0.032	8.328	0.000	0.206	0.333
Neighborhood_Somerst	0.2998	0.034	8.728	0.000	0.232	0.367
Neighborhood_StoneBr	0.0933	0.026	3.637	0.000	0.043	0.144
Condition1_Norm	0.0382	0.009	4.085	0.000	0.020	0.057
BldgType_2fmCon	0.0575	0.027	2.127	0.034	0.004	0.111
BldgType_Twnhs	-0.0573	0.021	-2.736	0.006	-0.098	-0.016
HouseStyle_2.5Fin	-0.1903	0.058	-3.273	0.001	-0.304	-0.076
HouseStyle_2.5Unf	0.1073	0.040	2.668	0.008	0.028	0.186
RoofStyle_Shed	0.3565	0.112	3.179	0.002	0.136	0.577
RoofMatl_Membran	0.4561	0.117	3.893	0.000	0.226	0.686
RoofMatl_Metal	0.3542	0.113	3.140	0.002	0.133	0.576
Exterior1st_BrkComm	-0.1967	0.076	-2.583	0.010	-0.346	-0.047
Exterior1st_BrkFace	0.0639	0.020	3.128	0.002	0.024	0.104
Exterior1st_HdBoard	0.7492	0.089	8.419	0.000	0.575	0.924
Exterior1st_MetalSd	0.4084	0.046	8.870	0.000	0.318	0.499
Exterior1st_Plywood	0.3800	0.044	8.551	0.000	0.293	0.467
Exterior1st_VinylSd	0.3820	0.046	8.305	0.000	0.292	0.472
Exterior1st_Wd Sdng	0.3590	0.044	8.221	0.000	0.273	0.445
Exterior2nd_HdBoard	0.7595	0.089	8.525	0.000	0.585	0.934
Exterior2nd_MetalSd	0.3614	0.046	7.877	0.000	0.271	0.451
Exterior2nd_Plywood	0.3923	0.045	8.758	0.000	0.304	0.480
Exterior2nd_VinylSd	0.3836	0.046	8.381	0.000	0.294	0.473
Exterior2nd_Wd Sdng	0.4026	0.044	9.130	0.000	0.316	0.489
Foundation_PConc	0.0324	0.010	3.165	0.002	0.012	0.052
Foundation_Stone	0.0957	0.043	2.248	0.025	0.012	0.179
Foundation_Wood	-0.1306	0.058	-2.239	0.025	-0.245	-0.016
BsmtCond_Gd	0.0810	0.024	3.403	0.001	0.034	0.128
BsmtCond_Po	0.1999	0.082	2.424	0.016	0.038	0.362
BsmtCond_TA	0.0768	0.019	3.955	0.000	0.039	0.115

```
BsmtExposure_Gd              0.0501      0.012      4.292      0.000       0.027       0.073
BsmtFinType1_GLQ             0.0274      0.009      2.930      0.003       0.009       0.046
BsmtFinType2_BLQ            -0.0885      0.021     -4.158      0.000      -0.130      -0.047
Heating_GasW                 0.0917      0.035      2.643      0.008       0.024       0.160
Heating_Grav                -0.2813      0.047     -5.953      0.000      -0.374      -0.189
HeatingQC_TA                -0.0230      0.008     -2.927      0.003      -0.038      -0.008
CentralAir_Y                 0.0489      0.016      3.035      0.002       0.017       0.081
KitchenQual_Fa              -0.0681      0.027     -2.487      0.013      -0.122      -0.014
KitchenQual_Gd              -0.0649      0.015     -4.335      0.000      -0.094      -0.036
KitchenQual_TA              -0.0653      0.018     -3.724      0.000      -0.100      -0.031
Functional_Maj2             -0.2874      0.050     -5.737      0.000      -0.386      -0.189
Functional_Mod              -0.1361      0.039     -3.519      0.000      -0.212      -0.060
Functional_Sev              -0.4002      0.100     -3.996      0.000      -0.597      -0.204
Functional_Typ               0.0461      0.014      3.209      0.001       0.018       0.074
GarageQual_Fa               -0.0385      0.019     -2.043      0.041      -0.075      -0.002
PavedDrive_Y                 0.0433      0.013      3.296      0.001       0.018       0.069
Fence_MnPrv                  0.0211      0.011      1.971      0.049     9.4e-05       0.042
SaleType_CWD                 0.1221      0.058      2.119      0.034       0.009       0.235
SaleType_Oth                 0.1629      0.071      2.282      0.023       0.023       0.303
SaleCondition_Normal         0.0593      0.011      5.320      0.000       0.037       0.081
SaleCondition_Partial        0.1125      0.016      6.914      0.000       0.081       0.144
Exterior_grp_1_MetalSd       0.4084      0.046      8.870      0.000       0.318       0.499
Exterior_grp_1_Other         0.7740      0.088      8.790      0.000       0.601       0.947
Exterior_grp_1_Plywood       0.3800      0.044      8.551      0.000       0.293       0.467
Exterior_grp_1_VinylSd       0.3820      0.046      8.305      0.000       0.292       0.472
Exterior_grp_1_Wd Sdng       0.3590      0.044      8.221      0.000       0.273       0.445
Exterior_grp_2_MetalSd       0.3614      0.046      7.877      0.000       0.271       0.451
Exterior_grp_2_Other         0.7533      0.087      8.636      0.000       0.582       0.924
Exterior_grp_2_Plywood       0.3923      0.045      8.758      0.000       0.304       0.480
Exterior_grp_2_VinylSd       0.3836      0.046      8.381      0.000       0.294       0.473
Exterior_grp_2_Wd Sdng       0.4026      0.044      9.130      0.000       0.316       0.489
Neighborhood_grp_Edwards     0.2567      0.032      8.071      0.000       0.194       0.319
Neighborhood_grp_Gilbert     0.2705      0.034      8.030      0.000       0.204       0.337
Neighborhood_grp_NAmes       0.2711      0.031      8.619      0.000       0.209       0.333
Neighborhood_grp_NridgHt     0.3133      0.033      9.567      0.000       0.249       0.378
Neighborhood_grp_OldTown     0.2647      0.031      8.473      0.000       0.203       0.326
Neighborhood_grp_Other       0.5576      0.063      8.865      0.000       0.434       0.681
Neighborhood_grp_Sawyer      0.2697      0.032      8.328      0.000       0.206       0.333
Neighborhood_grp_Somerst     0.2998      0.034      8.728      0.000       0.232       0.367
==============================================================================
Omnibus:                      164.452   Durbin-Watson:                   1.971
```

```
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              948.495
Skew:                          -0.514   Prob(JB):                    1.09e-206
Kurtosis:                       7.343   Cond. No.                     8.04e+18
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 7.3e-27. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

Model Summary

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          SalePrice_log   R-squared:                       0.941
Model:                            OLS   Adj. R-squared:                  0.937
Method:                 Least Squares   F-statistic:                     207.9
Date:                Sun, 15 Mar 2026   Prob (F-statistic):               0.00
Time:                        22:42:15   Log-Likelihood:                 1087.6
No. Observations:                1143   AIC:                            -2009.
Df Residuals:                    1060   BIC:                            -1591.
Df Model:                          82
Covariance Type:            nonrobust
==================================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------------------------
const                      3.0527      0.349      8.748      0.000       2.368       3.737
MSSubClass                -0.0002    5.8e-05     -2.763      0.006      -0.000   -4.65e-05
LotFrontage                0.0003   9.35e-05      2.914      0.004     8.9e-05       0.000
LotArea                 1.391e-06   2.28e-07      6.112      0.000    9.44e-07    1.84e-06
OverallQual                0.0230      0.002     10.683      0.000       0.019       0.027
OverallCond                0.0171      0.002      9.215      0.000       0.013       0.021
YearBuilt                  0.0008      0.000      6.595      0.000       0.001       0.001
YearRemodAdd               0.0005      0.000      3.769      0.000       0.000       0.001
BsmtUnfSF              -2.876e-05   5.53e-06     -5.205      0.000   -3.96e-05   -1.79e-05
TotalBsmtSF             7.281e-05   6.64e-06     10.963      0.000    5.98e-05    8.58e-05
GrLivArea                  0.0002   5.28e-06     28.607      0.000       0.000       0.000
BsmtFullBath               0.0131      0.004      3.149      0.002       0.005       0.021
KitchenAbvGr              -0.0464      0.008     -5.592      0.000      -0.063      -0.030
Fireplaces                 0.0096      0.003      3.221      0.001       0.004       0.015
GarageCars                 0.0232      0.003      8.209      0.000       0.018       0.029
WoodDeckSF              5.346e-05   1.31e-05      4.080      0.000    2.77e-05    7.92e-05
```

ScreenPorch	7.93e-05	2.97e-05	2.668	0.008	2.1e-05	0.000
PoolArea	0.0002	4.46e-05	3.370	0.001	6.28e-05	0.000
MSSubClass	-0.0002	5.8e-05	-2.763	0.006	-0.000	-4.65e-05
LotFrontage	0.0003	9.35e-05	2.914	0.004	8.9e-05	0.000
LotArea	1.391e-06	2.28e-07	6.113	0.000	9.45e-07	1.84e-06
OverallQual	0.0230	0.002	10.683	0.000	0.019	0.027
OverallCond	0.0171	0.002	9.215	0.000	0.013	0.021
YearBuilt	0.0008	0.000	6.595	0.000	0.001	0.001
YearRemodAdd	0.0005	0.000	3.769	0.000	0.000	0.001
BsmtUnfSF	-2.876e-05	5.53e-06	-5.205	0.000	-3.96e-05	-1.79e-05
TotalBsmtSF	7.281e-05	6.64e-06	10.963	0.000	5.98e-05	8.58e-05
GrLivArea	0.0002	5.28e-06	28.607	0.000	0.000	0.000
BsmtFullBath	0.0131	0.004	3.149	0.002	0.005	0.021
KitchenAbvGr	-0.0464	0.008	-5.592	0.000	-0.063	-0.030
Fireplaces	0.0096	0.003	3.221	0.001	0.004	0.015
GarageCars	0.0232	0.003	8.209	0.000	0.018	0.029
WoodDeckSF	5.346e-05	1.31e-05	4.080	0.000	2.77e-05	7.92e-05
ScreenPorch	7.93e-05	2.97e-05	2.668	0.008	2.1e-05	0.000
PoolArea	0.0002	4.46e-05	3.370	0.001	6.28e-05	0.000
MSZoning_FV	0.3964	0.047	8.417	0.000	0.304	0.489
MSZoning_RH	0.3703	0.048	7.755	0.000	0.277	0.464
MSZoning_RL	0.3757	0.038	9.802	0.000	0.300	0.451
MSZoning_RM	0.3362	0.039	8.690	0.000	0.260	0.412
LotConfig_CulDSac	0.0475	0.013	3.682	0.000	0.022	0.073
LandSlope_Sev	-0.2816	0.057	-4.964	0.000	-0.393	-0.170
Neighborhood_CollgCr	0.5493	0.065	8.398	0.000	0.421	0.678
Neighborhood_Crawfor	0.1176	0.018	6.564	0.000	0.082	0.153
Neighborhood_Edwards	0.2567	0.032	8.071	0.000	0.194	0.319
Neighborhood_Gilbert	0.2705	0.034	8.030	0.000	0.204	0.337
Neighborhood_MeadowV	-0.1258	0.034	-3.697	0.000	-0.193	-0.059
Neighborhood_Mitchel	-0.0478	0.018	-2.658	0.008	-0.083	-0.013
Neighborhood_NAmes	0.2711	0.031	8.619	0.000	0.209	0.333
Neighborhood_NridgHt	0.3133	0.033	9.567	0.000	0.249	0.378
Neighborhood_OldTown	0.2647	0.031	8.473	0.000	0.203	0.326
Neighborhood_Sawyer	0.2697	0.032	8.328	0.000	0.206	0.333
Neighborhood_Somerst	0.2998	0.034	8.728	0.000	0.232	0.367
Neighborhood_StoneBr	0.0933	0.026	3.637	0.000	0.043	0.144
Condition1_Norm	0.0382	0.009	4.085	0.000	0.020	0.057
BldgType_2fmCon	0.0575	0.027	2.127	0.034	0.004	0.111
BldgType_Twnhs	-0.0573	0.021	-2.736	0.006	-0.098	-0.016
HouseStyle_2.5Fin	-0.1903	0.058	-3.273	0.001	-0.304	-0.076

HouseStyle_2.5Unf	0.1073	0.040	2.668	0.008	0.028	0.186
RoofStyle_Shed	0.3565	0.112	3.179	0.002	0.136	0.577
RoofMatl_Membran	0.4561	0.117	3.893	0.000	0.226	0.686
RoofMatl_Metal	0.3542	0.113	3.140	0.002	0.133	0.576
Exterior1st_BrkComm	-0.1967	0.076	-2.583	0.010	-0.346	-0.047
Exterior1st_BrkFace	0.0639	0.020	3.128	0.002	0.024	0.104
Exterior1st_HdBoard	0.7492	0.089	8.419	0.000	0.575	0.924
Exterior1st_MetalSd	0.4084	0.046	8.870	0.000	0.318	0.499
Exterior1st_Plywood	0.3800	0.044	8.551	0.000	0.293	0.467
Exterior1st_VinylSd	0.3820	0.046	8.305	0.000	0.292	0.472
Exterior1st_Wd Sdng	0.3590	0.044	8.221	0.000	0.273	0.445
Exterior2nd_HdBoard	0.7595	0.089	8.525	0.000	0.585	0.934
Exterior2nd_MetalSd	0.3614	0.046	7.877	0.000	0.271	0.451
Exterior2nd_Plywood	0.3923	0.045	8.758	0.000	0.304	0.480
Exterior2nd_VinylSd	0.3836	0.046	8.381	0.000	0.294	0.473
Exterior2nd_Wd Sdng	0.4026	0.044	9.130	0.000	0.316	0.489
Foundation_PConc	0.0324	0.010	3.165	0.002	0.012	0.052
Foundation_Stone	0.0957	0.043	2.248	0.025	0.012	0.179
Foundation_Wood	-0.1306	0.058	-2.239	0.025	-0.245	-0.016
BsmtCond_Gd	0.0810	0.024	3.403	0.001	0.034	0.128
BsmtCond_Po	0.1999	0.082	2.424	0.016	0.038	0.362
BsmtCond_TA	0.0768	0.019	3.955	0.000	0.039	0.115
BsmtExposure_Gd	0.0501	0.012	4.292	0.000	0.027	0.073
BsmtFinType1_GLQ	0.0274	0.009	2.930	0.003	0.009	0.046
BsmtFinType2_BLQ	-0.0885	0.021	-4.158	0.000	-0.130	-0.047
Heating_GasW	0.0917	0.035	2.643	0.008	0.024	0.160
Heating_Grav	-0.2813	0.047	-5.953	0.000	-0.374	-0.189
HeatingQC_TA	-0.0230	0.008	-2.927	0.003	-0.038	-0.008
CentralAir_Y	0.0489	0.016	3.035	0.002	0.017	0.081
KitchenQual_Fa	-0.0681	0.027	-2.487	0.013	-0.122	-0.014
KitchenQual_Gd	-0.0649	0.015	-4.335	0.000	-0.094	-0.036
KitchenQual_TA	-0.0653	0.018	-3.724	0.000	-0.100	-0.031
Functional_Maj2	-0.2874	0.050	-5.737	0.000	-0.386	-0.189
Functional_Mod	-0.1361	0.039	-3.519	0.000	-0.212	-0.060
Functional_Sev	-0.4002	0.100	-3.996	0.000	-0.597	-0.204
Functional_Typ	0.0461	0.014	3.209	0.001	0.018	0.074
GarageQual_Fa	-0.0385	0.019	-2.043	0.041	-0.075	-0.002
PavedDrive_Y	0.0433	0.013	3.296	0.001	0.018	0.069
Fence_MnPrv	0.0211	0.011	1.971	0.049	9.4e-05	0.042
SaleType_CWD	0.1221	0.058	2.119	0.034	0.009	0.235
SaleType_Oth	0.1629	0.071	2.282	0.023	0.023	0.303

```
SaleCondition_Normal         0.0593      0.011      5.320      0.000       0.037       0.081
SaleCondition_Partial        0.1125      0.016      6.914      0.000       0.081       0.144
Exterior_grp_1_MetalSd       0.4084      0.046      8.870      0.000       0.318       0.499
Exterior_grp_1_Other         0.7740      0.088      8.790      0.000       0.601       0.947
Exterior_grp_1_Plywood       0.3800      0.044      8.551      0.000       0.293       0.467
Exterior_grp_1_VinylSd       0.3820      0.046      8.305      0.000       0.292       0.472
Exterior_grp_1_Wd Sdng       0.3590      0.044      8.221      0.000       0.273       0.445
Exterior_grp_2_MetalSd       0.3614      0.046      7.877      0.000       0.271       0.451
Exterior_grp_2_Other         0.7533      0.087      8.636      0.000       0.582       0.924
Exterior_grp_2_Plywood       0.3923      0.045      8.758      0.000       0.304       0.480
Exterior_grp_2_VinylSd       0.3836      0.046      8.381      0.000       0.294       0.473
Exterior_grp_2_Wd Sdng       0.4026      0.044      9.130      0.000       0.316       0.489
Neighborhood_grp_Edwards     0.2567      0.032      8.071      0.000       0.194       0.319
Neighborhood_grp_Gilbert     0.2705      0.034      8.030      0.000       0.204       0.337
Neighborhood_grp_NAmes       0.2711      0.031      8.619      0.000       0.209       0.333
Neighborhood_grp_NridgHt     0.3133      0.033      9.567      0.000       0.249       0.378
Neighborhood_grp_OldTown     0.2647      0.031      8.473      0.000       0.203       0.326
Neighborhood_grp_Other       0.5576      0.063      8.865      0.000       0.434       0.681
Neighborhood_grp_Sawyer      0.2697      0.032      8.328      0.000       0.206       0.333
Neighborhood_grp_Somerst     0.2998      0.034      8.728      0.000       0.232       0.367
==============================================================================
Omnibus:                      164.452   Durbin-Watson:                   1.971
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              948.495
Skew:                          -0.514   Prob(JB):                    1.09e-206
Kurtosis:                       7.343   Cond. No.                     8.04e+18
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 7.3e-27. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

Model Diagnostics

Heteroskedasticity Test

In [102…

```
bp_test = het_breuschpagan(model.resid, model.model.exog)

bp_labels = [
```

```
    "Lagrange multiplier statistic",
    "p-value",
    "f-value",
    "f p-value"
]

print(dict(zip(bp_labels, bp_test)))
```

```
{'Lagrange multiplier statistic': np.float64(220.33692551801735), 'p-value': np.float64(2.4912799404681143e-08), 'f-value': np.
float64(3.08699664747183), 'f p-value': np.float64(9.706036567418119e-17)}
```

Normality Test

In [103…

```
jb_test = jarque_bera(model.resid)

print("Jarque-Bera:", jb_test)
```

```
Jarque-Bera: SignificanceResult(statistic=np.float64(948.4948177520433), pvalue=np.float64(1.0888480262814167e-206))
```

Ramsey RESET Test

In [104…

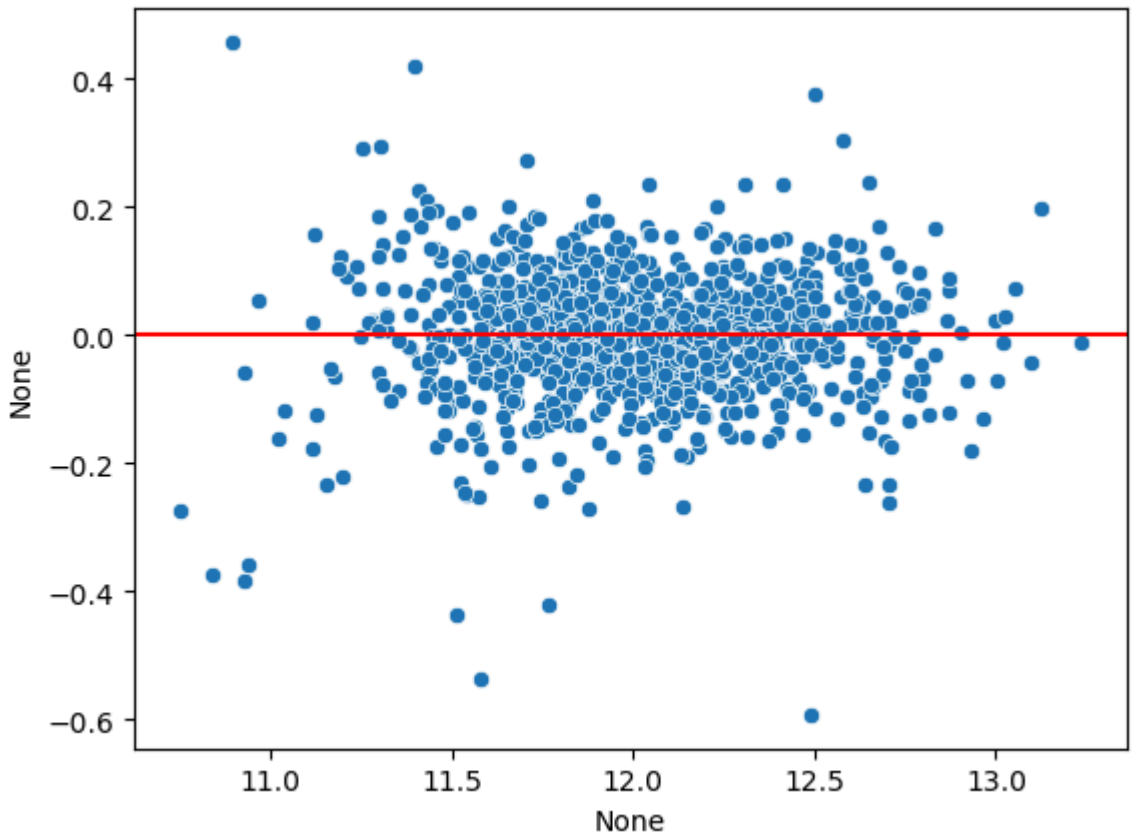
```
reset = linear_reset(model, power=2, use_f=True)

print(reset)
```

```
<F test: F=5.834924747575591, p=0.015879546357865357, df_denom=1.06e+03, df_num=1>
```

Residual plot

In [105…

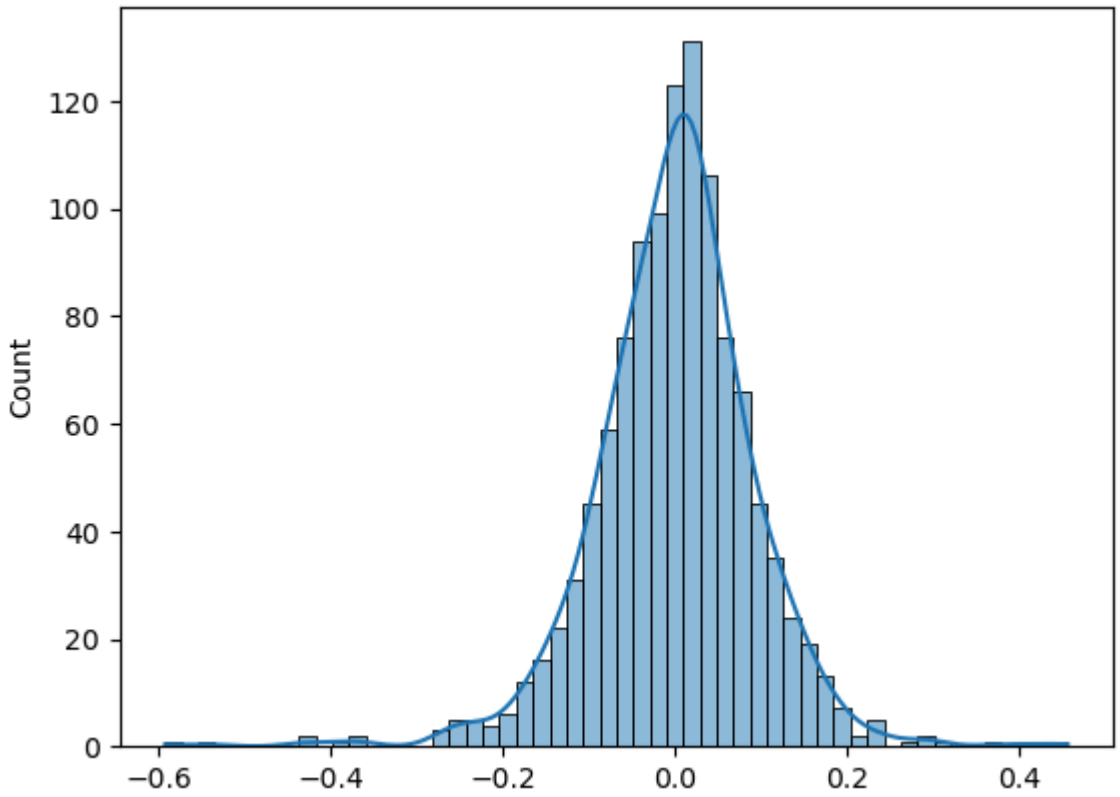
```
residuals = model.resid

sns.scatterplot(x=model.fittedvalues, y=residuals)

plt.axhline(0,color="red")

plt.show()
```



Normality of residuals

In [106…

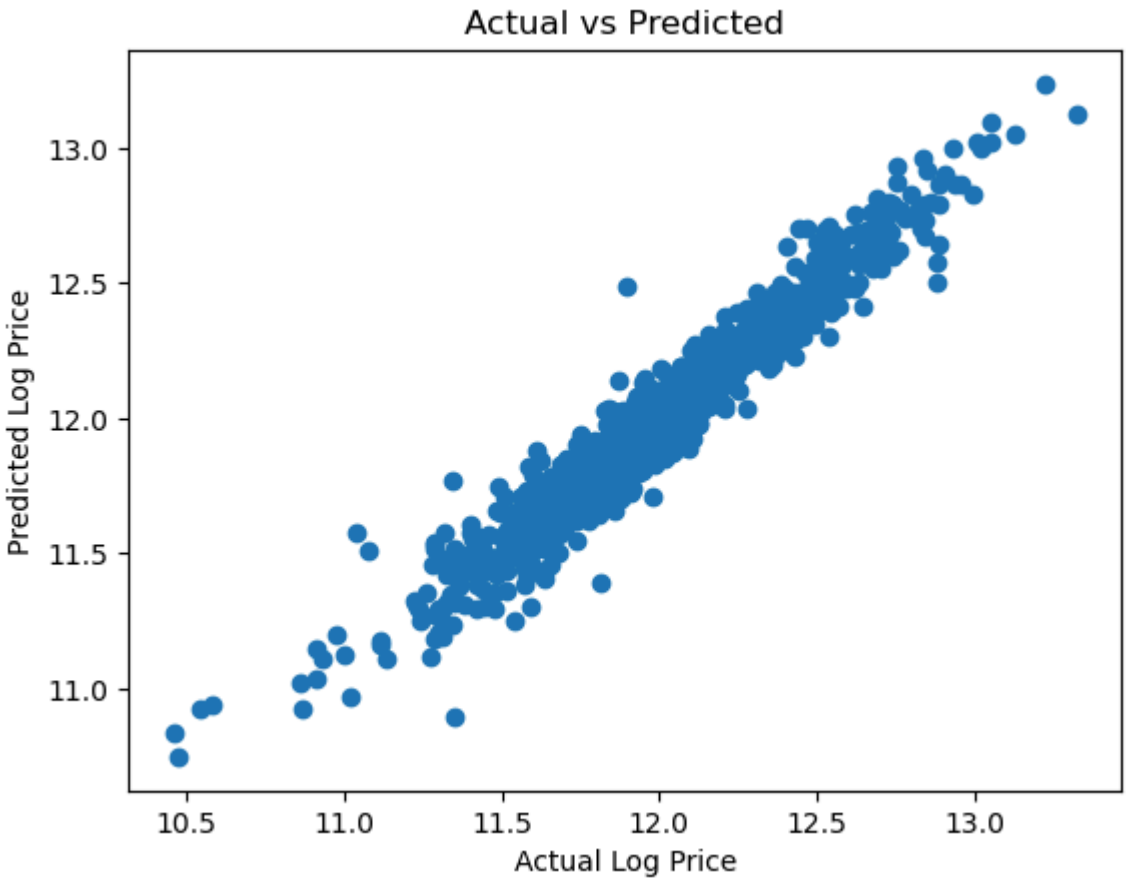
```
sns.histplot(residuals, kde=True)

plt.show()
```



Predicted vs Actual

In [107…

```
pred = model.predict(X_final)


plt.scatter(y_train, pred)

plt.xlabel("Actual Log Price")

plt.ylabel("Predicted Log Price")

plt.title("Actual vs Predicted")

plt.show()
```



Prepare Test Data

In [108…

```
# numeric variables used after VIF
test_subset = X_test[X_test.columns.intersection(X_vif.columns)]

# encode only categorical variables
test_cat = pd.get_dummies(
    X_test.select_dtypes(include="object"),
    drop_first=True
)
```

```
# combine numeric + categorical
test_sample = pd.concat([test_subset, test_cat], axis=1)

# align columns with training model
X_test_final = test_sample.reindex(
    columns=X_final.columns.drop("const"),
    fill_value=0
)

# add constant
X_test_final = sm.add_constant(X_test_final)
```

Predict on Test Data

In [109…

```
y_pred = model.predict(X_test_final)
```

Prediction Error Metrics

In [110…

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", rmse)
print("Test MAE:", mae)
```

```
Test RMSE: 0.12427230254410745
Test MAE: 0.08449062616253147
```

In [111…

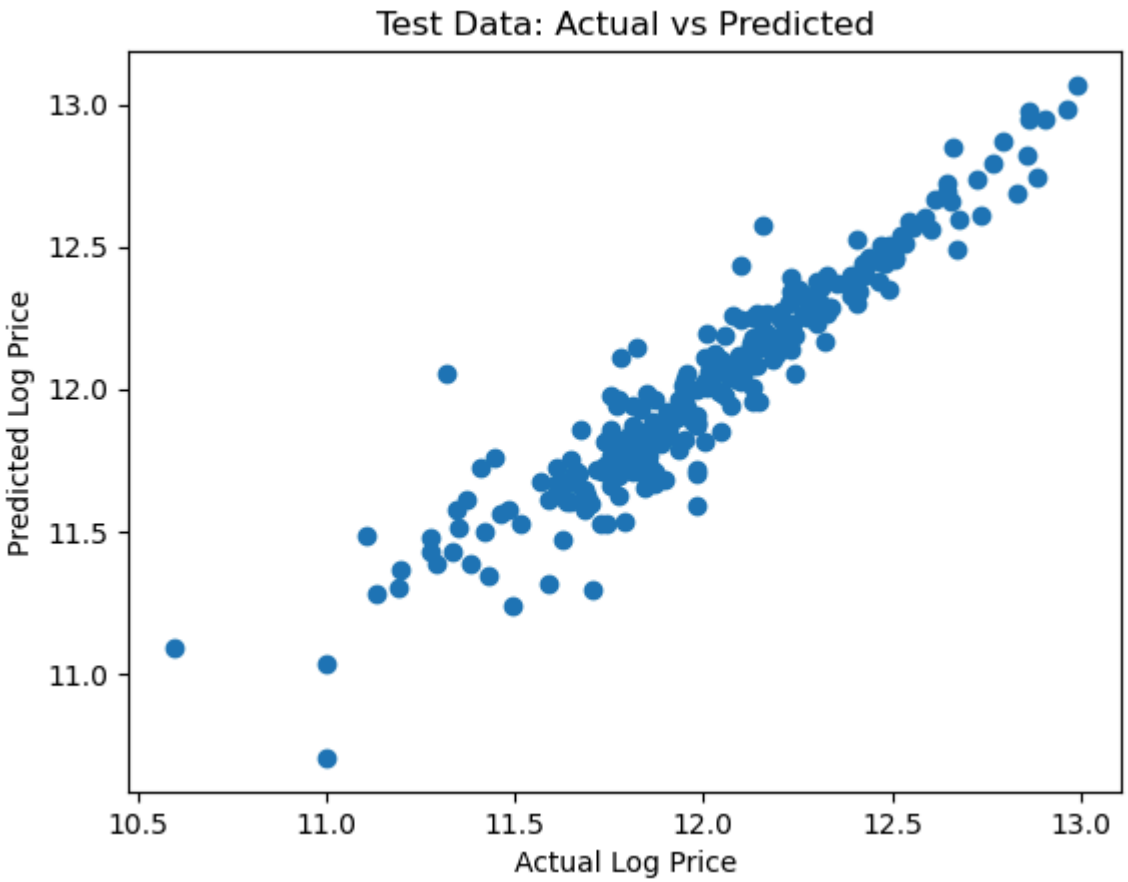
```
y_pred_ols=y_pred
```

Actual vs Predicted Plot

In [112…

```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
```

```
plt.title("Test Data: Actual vs Predicted")

plt.show()
```



Random Forest Model

In [113…

```
from sklearn.ensemble import RandomForestRegressor
# Remove constant column if present
X_train_model = X.drop(columns=["const"], errors="ignore")
```

```
rf_model = RandomForestRegressor(
    n_estimators=500,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_model, y_train)
```

Out[113…

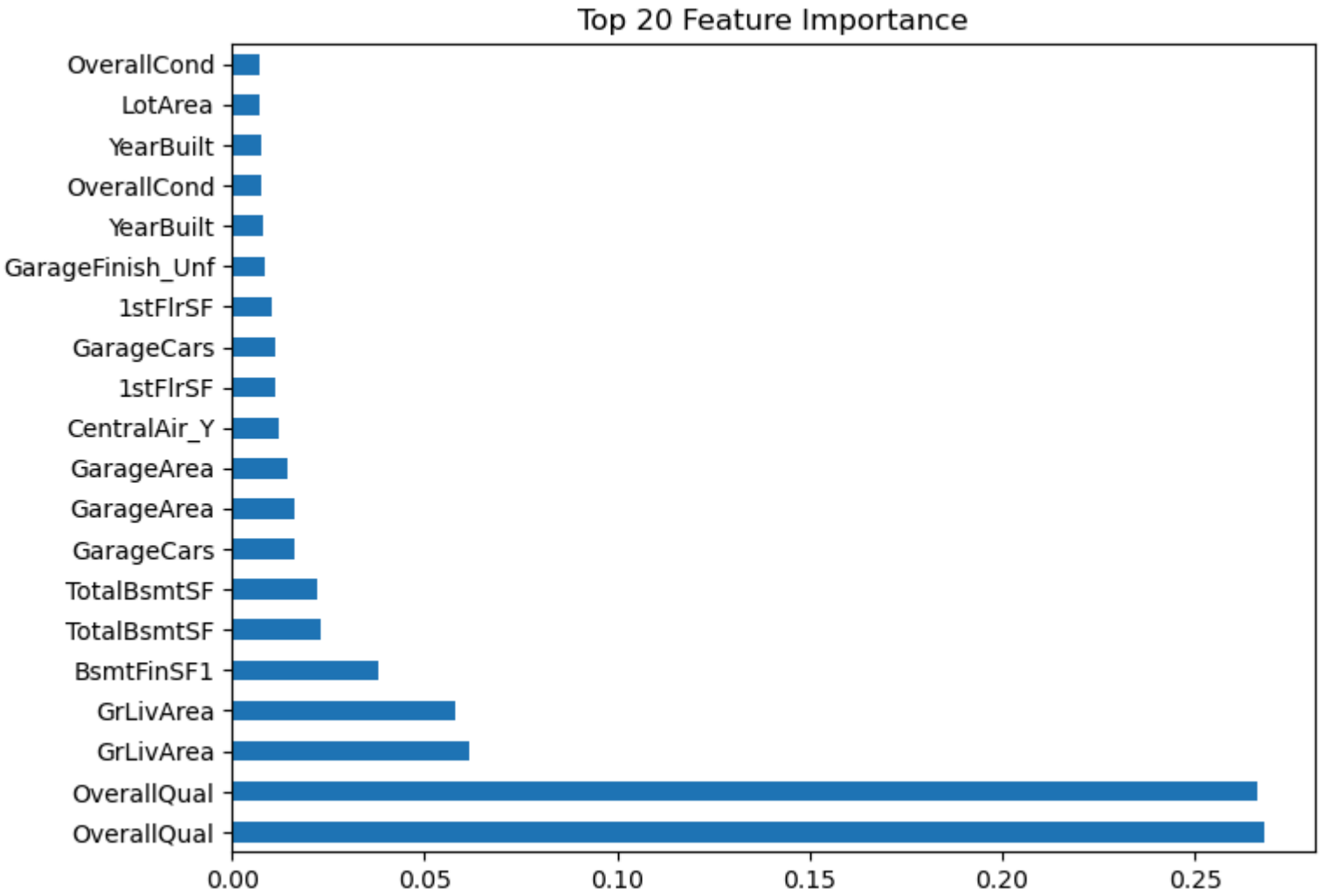
```
▾ RandomForestRegressor

▶ Parameters
```

Feature Importance

In [114…

```
feature_imp = pd.Series(
    rf_model.feature_importances_,
    index=X_train_model.columns
).sort_values(ascending=False)

print(feature_imp.head(20))

plt.figure(figsize=(8,6))
feature_imp.head(20).plot(kind="barh")
plt.title("Top 20 Feature Importance")
plt.show()
```

```
OverallQual         0.267932
OverallQual         0.266146
GrLivArea           0.061434
GrLivArea           0.058165
BsmtFinSF1          0.037917
TotalBsmtSF         0.022914
TotalBsmtSF         0.022247
GarageCars          0.016207
GarageArea          0.016124
GarageArea          0.014573
CentralAir_Y        0.012365
1stFlrSF            0.011236
GarageCars          0.011204
1stFlrSF            0.010392
GarageFinish_Unf    0.008704
YearBuilt           0.008088
OverallCond         0.007830
YearBuilt           0.007396
LotArea             0.006995
OverallCond         0.006983
dtype: float64
```

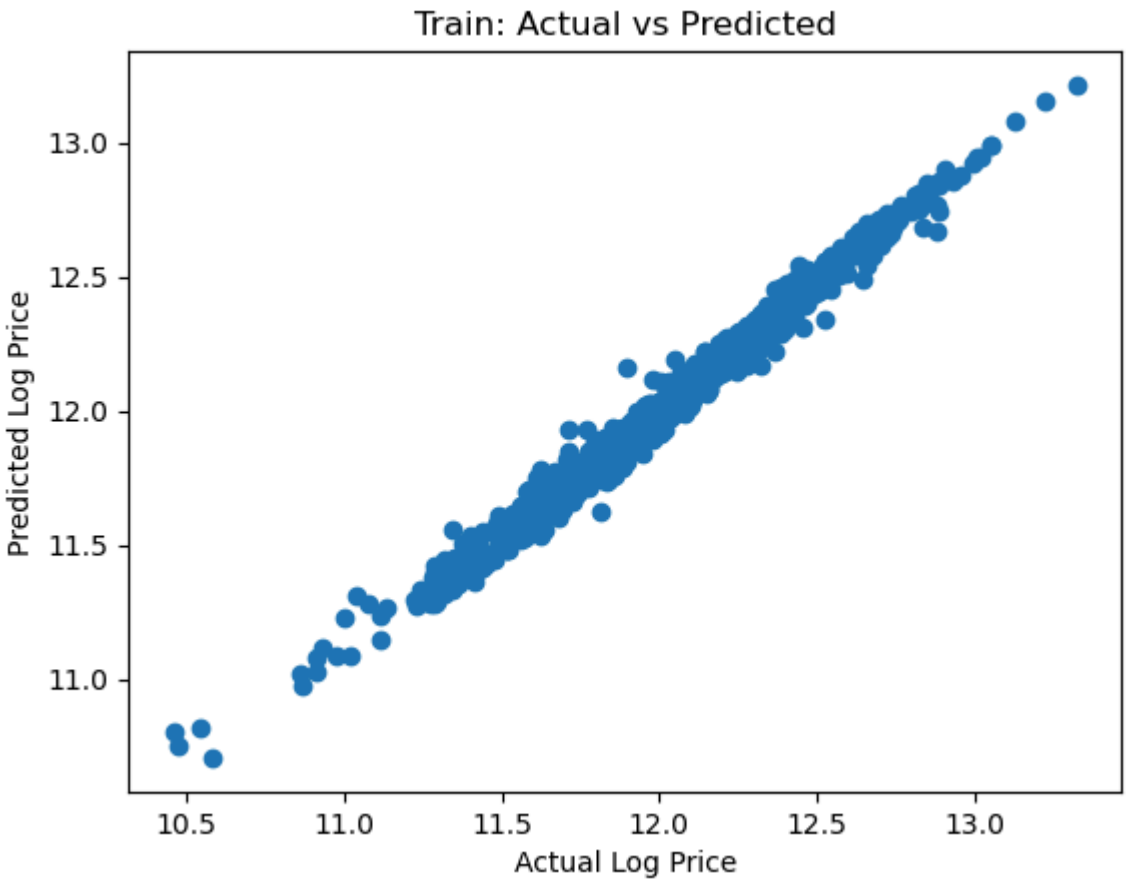


Prediction on Train Data

In [115…

```
train_pred = rf_model.predict(X_train_model)

plt.scatter(y_train, train_pred)
plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
```

```
plt.title("Train: Actual vs Predicted")
plt.show()
```



Prepare Test Data

In [116…

```
test_subset = X_test[X_test.columns.intersection(X_vif.columns)]

test_cat = pd.get_dummies(
    X_test.select_dtypes(include="object"),
    drop_first=True
)
```

```
test_sample = pd.concat([test_subset, test_cat], axis=1)

X_test_final = test_sample.reindex(
    columns=X_train_model.columns,
    fill_value=0
)
```

Predict on Test Data

In [117…

```
y_pred = rf_model.predict(X_test_final)
```

In [118…

```
y_pred_rf=y_pred
```

Model Evaluation

In [119…

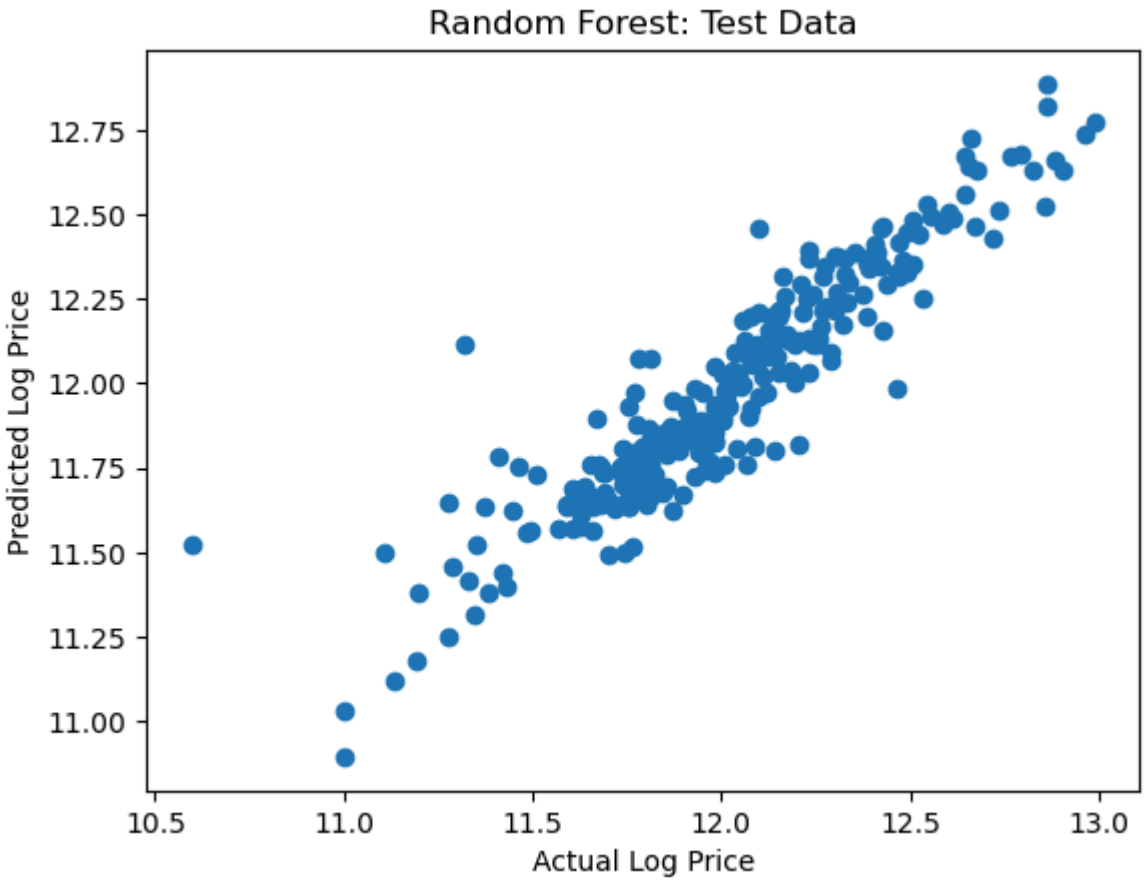
```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", rmse)
print("Test MAE:", mae)
```

```
Test RMSE: 0.14731897514940542
Test MAE: 0.10009781689940453
```

Actual vs Predicted Plot

In [120…

```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Random Forest: Test Data")

plt.show()
```



XGBoost Model

In [121…

```
from xgboost import XGBRegressor


# remove constant column
X_train_model = X_final.drop(columns=["const"], errors="ignore")

# ensure numeric
X_train_model = X_train_model.astype(float)
```

```
# convert to numpy (important)
X_train_model = X_train_model.values

xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

xgb_model.fit(X_train_model, y_train)
```

Out[121…

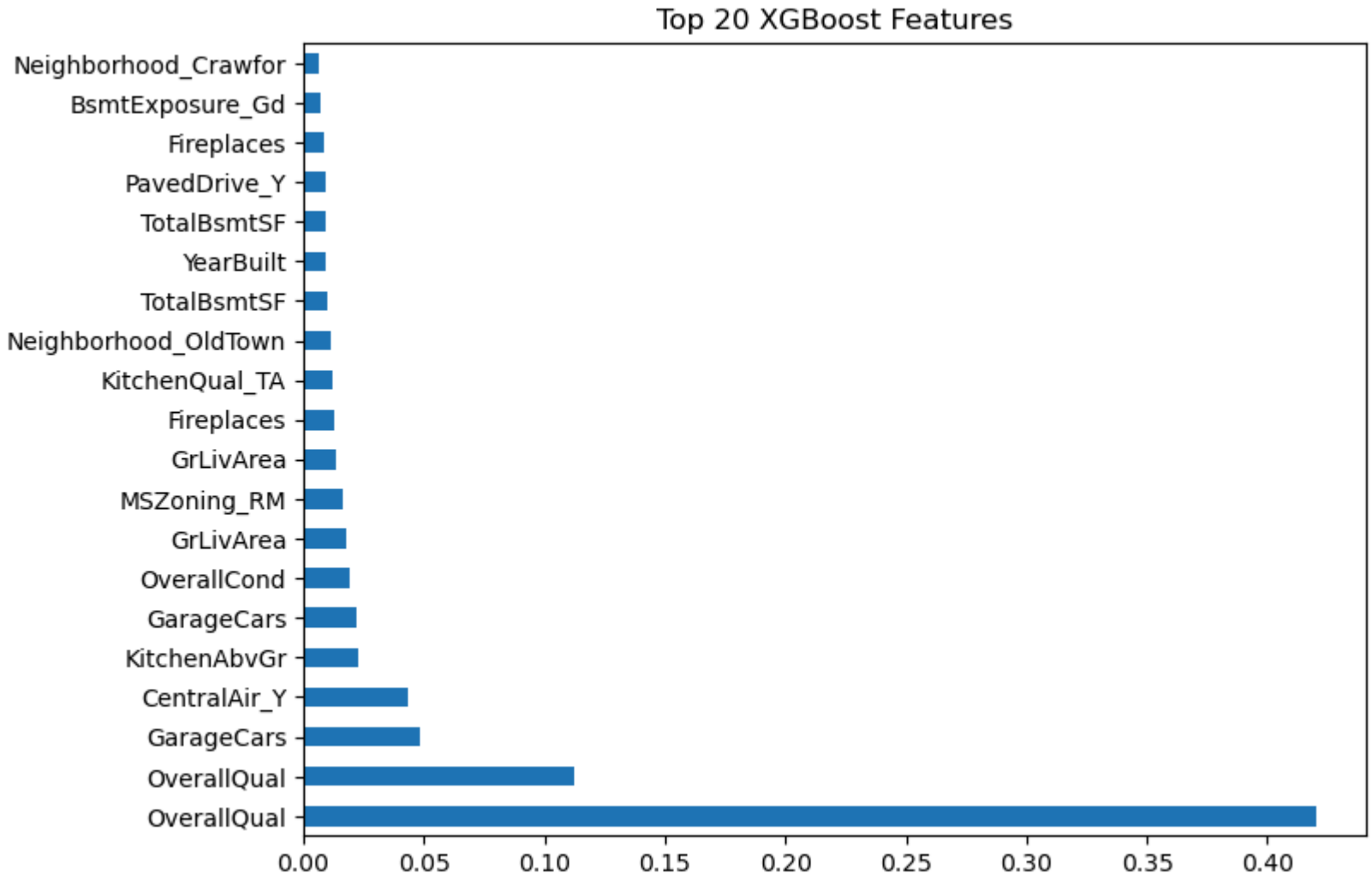
```
▾ XGBRegressor
▶ Parameters
```

Feature Importance

In [122…

```
#Save feature names before conversion
feature_names = X_final.drop(columns=["const"], errors="ignore").columns
importance = pd.Series(
    xgb_model.feature_importances_,
    index=feature_names
).sort_values(ascending=False)

print(importance.head(20))

plt.figure(figsize=(8,6))
importance.head(20).plot(kind="barh")
plt.title("Top 20 XGBoost Features")
plt.show()
```

```
OverallQual             0.419985
OverallQual             0.112463
GarageCars              0.048331
CentralAir_Y            0.043366
KitchenAbvGr            0.022666
GarageCars              0.021678
OverallCond             0.018919
GrLivArea               0.017638
MSZoning_RM             0.016041
GrLivArea               0.013734
Fireplaces              0.013007
KitchenQual_TA          0.012026
Neighborhood_OldTown    0.011608
TotalBsmtSF             0.010095
YearBuilt               0.009025
TotalBsmtSF             0.008877
PavedDrive_Y            0.008866
Fireplaces              0.008394
BsmtExposure_Gd         0.007245
Neighborhood_Crawfor    0.006572
dtype: float32
```

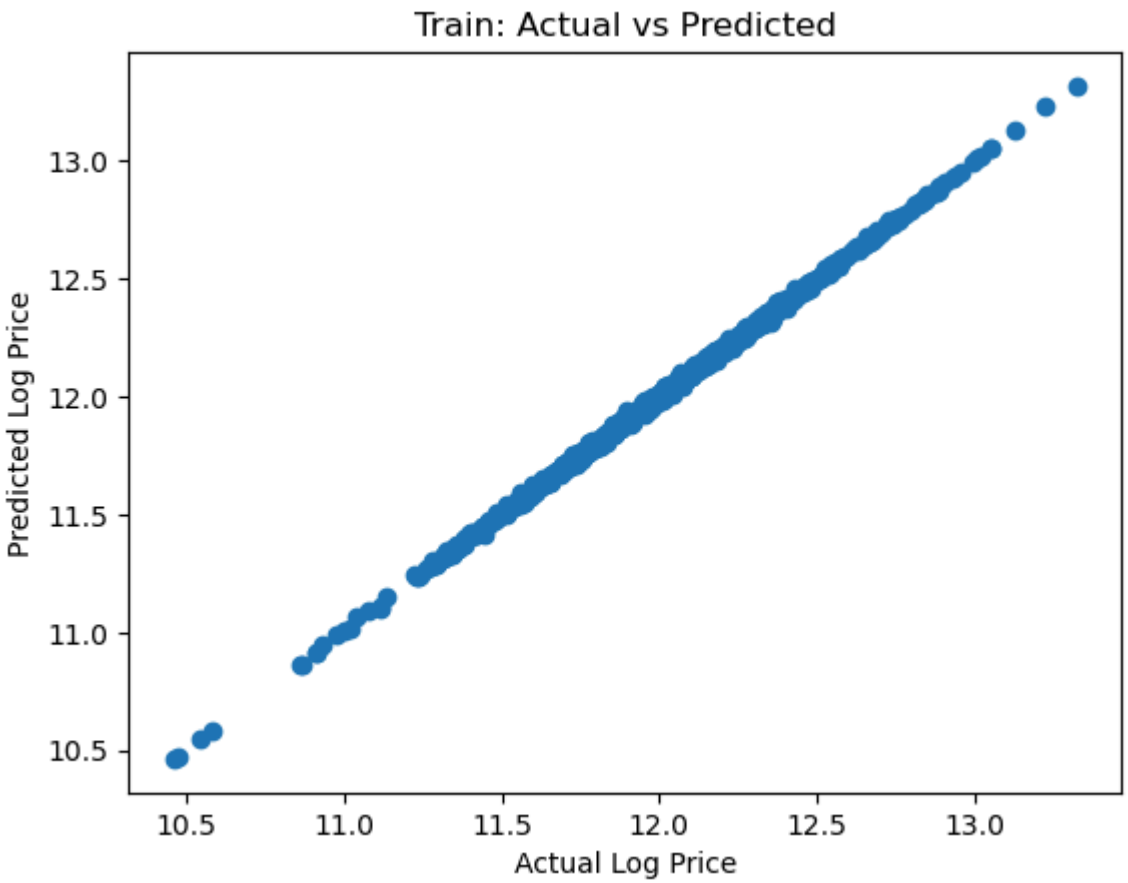


Train Prediction Plot

In [123…

```
train_pred = xgb_model.predict(X_train_model)

plt.scatter(y_train, train_pred)

plt.xlabel("Actual Log Price")
```

```
plt.ylabel("Predicted Log Price")

plt.title("Train: Actual vs Predicted")

plt.show()
```



Prepare Test Data

In [124…

```
test_subset = X_test[X_test.columns.intersection(X_vif.columns)]

test_cat = pd.get_dummies(
    X_test.select_dtypes(include="object"),
```

```
    drop_first=True
)

test_sample = pd.concat([test_subset, test_cat], axis=1)

X_test_final = test_sample.reindex(
    columns=feature_names,
    fill_value=0
)

#Convert to numeric before prediction


X_test_final = X_test_final.astype(float)
X_test_final = X_test_final.values
```

Predict on Test Data

In [125…

```
y_pred = xgb_model.predict(X_test_final)
```

In [126…

```
y_pred_xgb=y_pred
```

Evaluation

In [127…

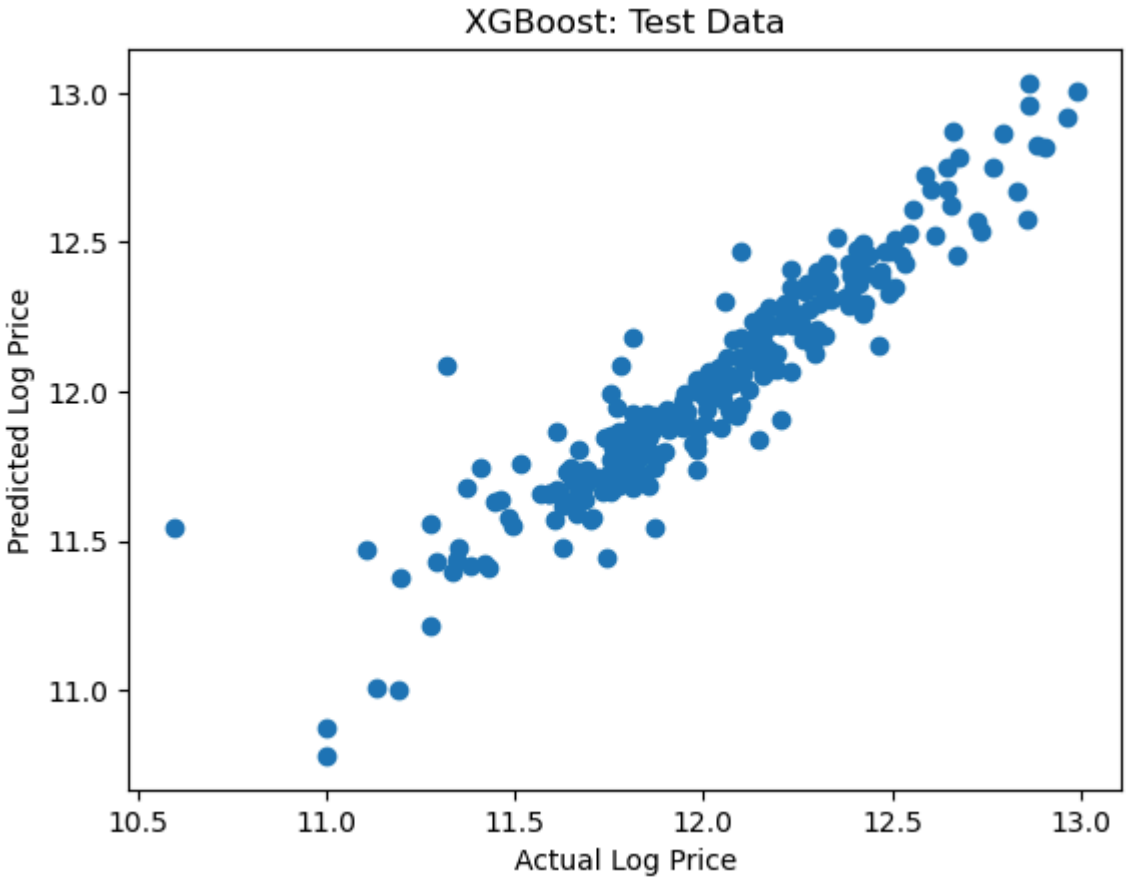
```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", rmse)
print("Test MAE:", mae)
```

```
Test RMSE: 0.13253483649405082
Test MAE: 0.08756973762976661
```

Actual vs Predicted Plot

In [128…

```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")

plt.title("XGBoost: Test Data")

plt.show()
```



Regularized Models

Random Forest with Regularization (Hyperparameter Tuning)

In [129…

```
from sklearn.model_selection import RandomizedSearchCV

# Remove constant column
X_train_rf = X_final.drop(columns=["const"], errors="ignore")

# Parameter grid for regularization
rf_param_grid = {
    "n_estimators": [300, 500, 700],
    "max_depth": [5, 8, 12, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}

rf_base = RandomForestRegressor(random_state=42)

rf_random = RandomizedSearchCV(
    estimator=rf_base,
    param_distributions=rf_param_grid,
    n_iter=20,
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    random_state=42
)

rf_random.fit(X_train_rf, y_train)

print("Best Parameters:", rf_random.best_params_)
```

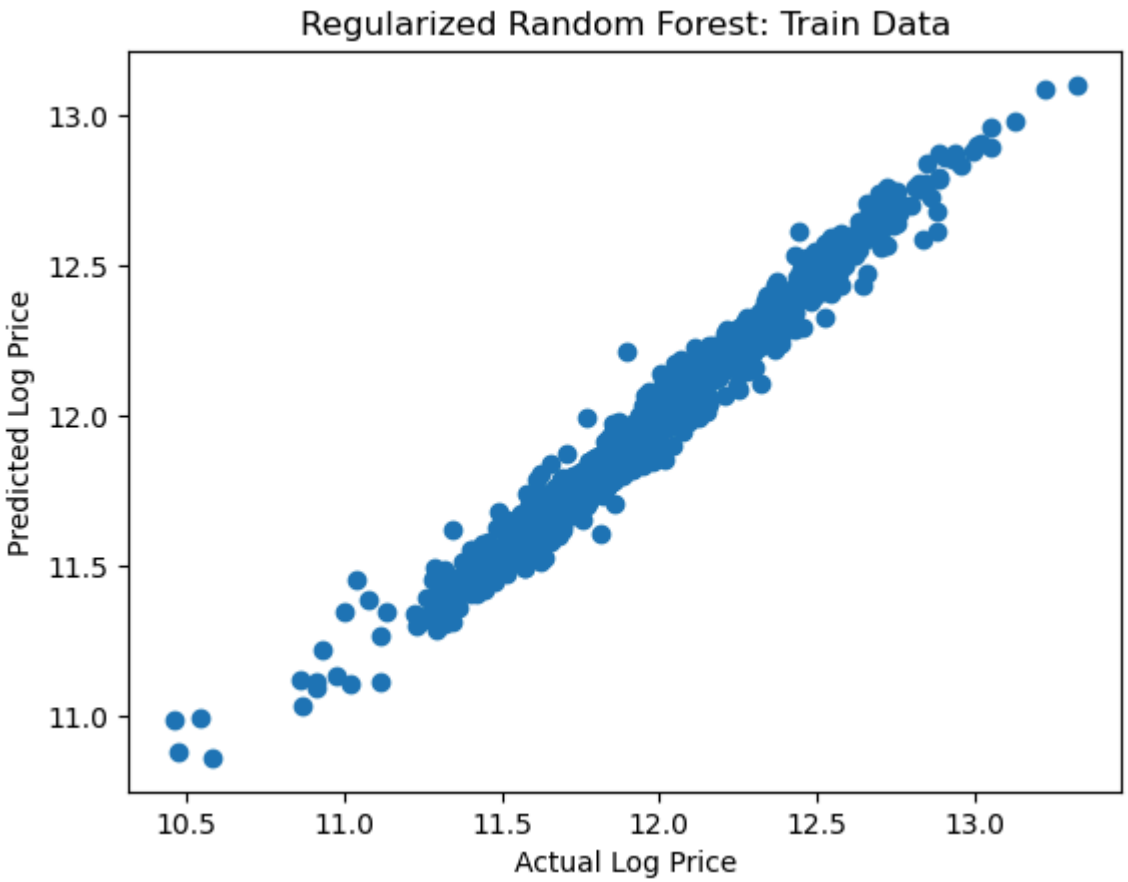
```
Best Parameters: {'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 12}
```

Train Best Regularized Random Forest

In [130…

```
rf_reg_model = rf_random.best_estimator_
```

```
train_pred = rf_reg_model.predict(X_train_rf)

plt.scatter(y_train, train_pred)
plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Regularized Random Forest: Train Data")
plt.show()
```



Predict on Test Data

In [131…

```
X_test_rf = X_test_final.copy()
```

```
y_pred_rf_reg = rf_reg_model.predict(X_test_rf)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf_reg))
mae = mean_absolute_error(y_test, y_pred_rf_reg)

print("Regularized RF Test RMSE:", rmse)
print("Regularized RF Test MAE:", mae)
```

```
Regularized RF Test RMSE: 0.13327325337697507
Regularized RF Test MAE: 0.08803193296308057
```

XGBoost with Regularization

The XGBoost model supports built-in regularization:

reg_alpha → L1 regularization

reg_lambda → L2 regularization

Hyperparameter Tuning for Regularized XGBoost

In [132…

```
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBRegressor

# Prepare training data
X_train_xgb = X_final.drop(columns=["const"], errors="ignore")

# Ensure numeric
X_train_xgb = X_train_xgb.astype(float)

# Convert to numpy
X_train_xgb = X_train_xgb.values
y_train_xgb = y_train.values

# Parameter grid
xgb_param_grid = {
    "n_estimators": [300, 500, 700],
    "max_depth": [3,4,5,6],
    "learning_rate": [0.01,0.03,0.05],
```

```
    "subsample": [0.7,0.8,0.9],
    "colsample_bytree": [0.7,0.8,0.9],
    "reg_alpha": [0,0.1,0.5,1],
    "reg_lambda": [0.5,1,2]
}

xgb_base = XGBRegressor(
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xgb_random = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=xgb_param_grid,
    n_iter=20,
    cv=5,
    scoring="neg_root_mean_squared_error",
    verbose=1,
    random_state=42,
    n_jobs=-1
)

# Fit model
xgb_random.fit(X_train_xgb, y_train_xgb)

print("Best Parameters:", xgb_random.best_params_)
```

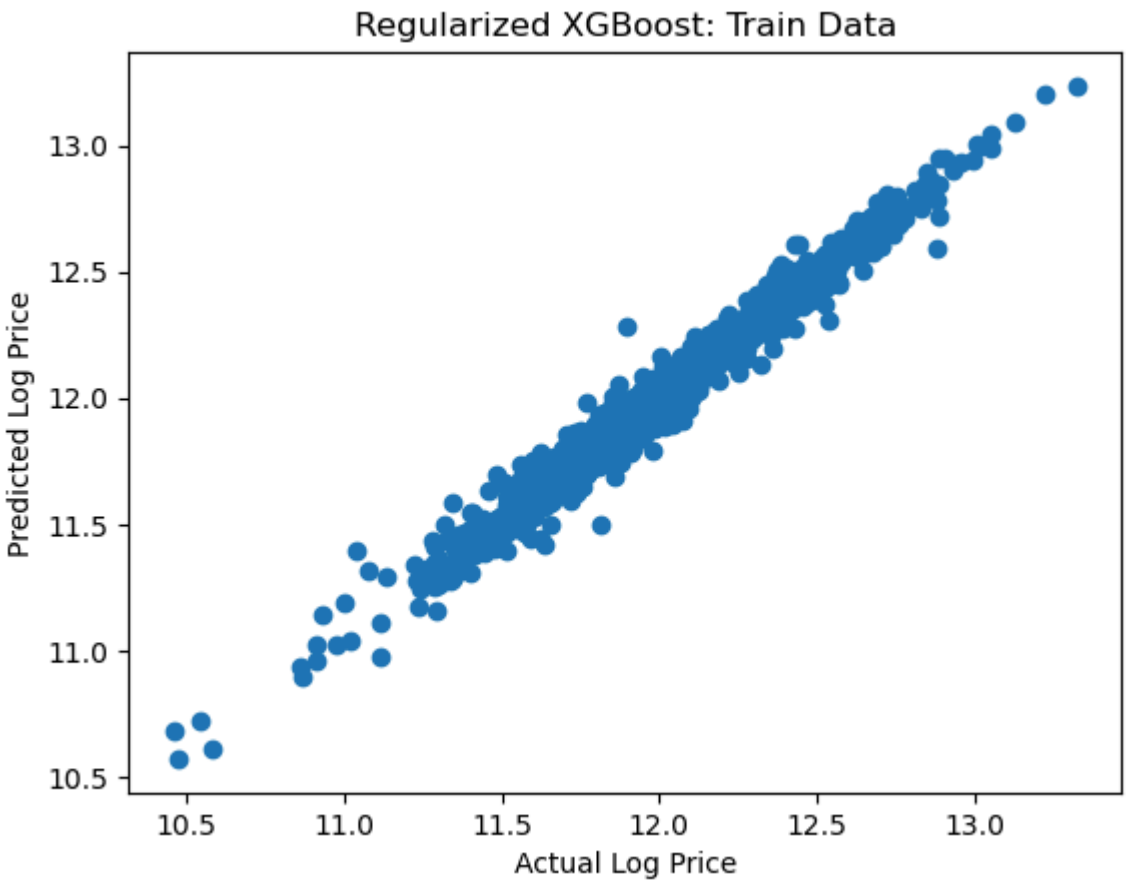
```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best Parameters: {'subsample': 0.7, 'reg_lambda': 2, 'reg_alpha': 0.1, 'n_estimators': 500, 'max_depth': 3, 'learning_rate': 0.
05, 'colsample_bytree': 0.9}
```

Train Best Regularized XGBoost Model

In [133…

```
xgb_reg_model = xgb_random.best_estimator_

train_pred = xgb_reg_model.predict(X_train_xgb)

plt.scatter(y_train, train_pred)
```

```
plt.xlabel("Actual Log Price")
plt.ylabel("Predicted Log Price")
plt.title("Regularized XGBoost: Train Data")

plt.show()
```



Predict on Test Data

In [134…

```
X_test_xgb = X_test_final.astype(float)

y_pred_xgb_reg = xgb_reg_model.predict(X_test_xgb)
```

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred_xgb_reg))
mae = mean_absolute_error(y_test, y_pred_xgb_reg)

print("Regularized XGBoost Test RMSE:", rmse)
print("Regularized XGBoost Test MAE:", mae)
```

```
Regularized XGBoost Test RMSE: 0.12193622657294091
Regularized XGBoost Test MAE: 0.07995580701058254
```

Model Comparison

In [135…

```
rmse_ols = np.sqrt(mean_squared_error(y_test, y_pred_ols))
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))

rmse_rf_reg = np.sqrt(mean_squared_error(y_test, y_pred_rf_reg))
rmse_xgb_reg = np.sqrt(mean_squared_error(y_test, y_pred_xgb_reg))

results = pd.DataFrame({
    "Model":["OLS","Random Forest","XGBoost","Random Forest_Regularized","XGBoost_Regularized"],
    "RMSE":[rmse_ols, rmse_rf, rmse_xgb,rmse_rf_reg,rmse_xgb_reg]
})
```

In [136…

```
results
```

Out[136…

	Model	RMSE
0	OLS	0.124272
1	Random Forest	0.147319
2	XGBoost	0.132535
3	Random Forest_Regularized	0.133273
4	XGBoost_Regularized	0.121936